

基于分步优化与多层模拟退火的农作物最优种植策略研究

摘要

当今，我国人均耕地面积远低于世界平均水平，因此优化种植策略对提升产量效益、降低风险和推动乡村振兴至关重要。基于这一背景，本团队创新性地建立了**分步优化模型**和**多层模拟退火算法**，在**满足约束条件**的同时，快速求解 2024 年-2030 年的**最优种植策略**。

问题一：我们采用**两步模拟退火算法**构建**分步优化模型**。首先，区分出种植方案相对固定的作物。然后，对剩余作物进行分类，通过**第一次模拟退火**确定各土地类型上各个作物最优种植面积。最后，以耕地分割次数最小化为目标，通过**第二次模拟退火**得到最优种植策略。这样分步处理，不仅减少计算量，还能灵活应对约束，避免局部最优陷阱。

问题二：在问题一基础上加入了农作物销售量、产量、种植成本和价格的**不确定性**。首先，设置变量的变化趋势并引入**随机扰动**，生成训练集，将问题一中第一步模拟退火的目标函数从单一输入量的最大利润调整为训练集的**平均利润最大化**。随后，生成测试集计算利润，通过 CV 值和最大回撤评估方案的抗风险能力，制定最优种植规划。

问题三：在问题二的基础上，考虑各数据波动之间的**相关性**。首先，通过计算 Spearman 相关系数，结合经济学原理确定需求量随价格变动公式。根据价格与种植成本之间的线性相关性，用成本变动预测价格和销售量。然后借鉴单因子模型，我们将成本变动分为宏观和个体因素并进行**随机模拟**。最终，完善的新方案不仅在平均利润上优于问题二的最优方案，还有效降低了风险。

关键字： 分步优化模型 多层模拟退火算法 最优种植策略 随机扰动

一、问题重述

1.1 问题背景

为助力乡村振兴，实现土地资源最优化配置，本团队以华北山区某乡村为例，展开了对农作物种植策略研究。该乡村现有露天耕地 1201 亩，包括平旱地、梯田、山坡地和水浇地 4 种类型，分散为 34 个大小不同的地块；另有 16 个普通大棚、4 个智慧大棚，每个大棚耕地面积为 0.6 亩。平旱地、梯田和山坡地适合每年种植一季粮食作物，水浇地适合每年种植一季水稻或两季蔬菜。普通大棚可种植一季蔬菜和一季食用菌，智慧大棚则适合种植两季蔬菜。为保证土壤肥力，每块地（含大棚）需避免重茬种植，且每三年内至少种植一次豆类作物。此外，种植策略需考虑方便耕作和田间管理，避免作物种植地过于分散，单块地面积过小。

附件一给出该乡村现有耕地情况以及农作物种类；附件二给出 2023 年农作物种植情况以及亩产量、种植成本、单价等销售数据。

1.2 问题要求

本题要求基于背景资料和附件数据，解决以下问题：

问题 1：假设附件 2 中提供的 2023 年相关数据保持不变，且所有农作物在当季生产并在当季销售。在以下两种情境下，建立数学模型，求解该乡村 2024-2030 年的最优农作物种植方案：

- i. 超过需求部分不能销售；
- ii. 超过需求部分按 2023 年销售价格的 50% 进行处理。

问题 2：在问题 1 的基础上，考虑农作物预期销售量、产量、种植成本及销售价格的不确定性，制定 2024-2030 年农作物的最优种植方案。具体要求如下：

- i. 预期销售量与产量的年变化范围为 $\pm 5\%$ 至 $\pm 10\%$ ；
- ii. 种植成本每年平均增长 5%；
- iii. 粮食类作物价格保持稳定，蔬菜类作物价格年均增长约 5%，食用菌类作物价格每年下降 1% 至 5%。

问题 3：在问题 2 的基础上，综合考虑多方面因素，制定 2024-2030 年的最优农作物种植策略。在求解最优种植方案时，需结合以下实际情况：

- i. 考虑不同作物之间的替代性和相关性；
- ii. 结合预期销售量、销售价格与种植成本三者之间的相互影响。

此外，**问题 1、2、3** 中的最优种植方案需满足以下约束条件

- i. 重茬限制：同一地块（含大棚）作物不可连续种植。

- ii. 豆类要求：每地块三年内至少种一次豆类。
- iii. 作业便利：作物种植不宜过于分散，单块面积不宜过小。

二、问题分析

2.1 问题一分析

对于问题一，我们采用分步优化模型，并通过两步模拟退火算法提高计算效率。首先，对各类数据进行分析，理解其现实意义，并将需求和种植方案相对固定的作物剥离出来，以简化问题。接着，根据种植特点和约束条件对剩余作物进行分类，这些分类之间相对独立。然后，我们以最大化总利润为目标，使用模拟退火算法确定各类作物在不同耕地类型上的种植面积。最后，我们进行第二步优化，以最小化耕地分割和切换种植次数为目标，再次应用模拟退火算法，得出最终的最优种植方案。

2.2 问题二分析

对于问题二，在问题一的基础上加入风险因素。我们确定各输入量公式以符合实际变化趋势，并加入随机扰动项以模拟现实生活中各数据量的不确定性以及潜在的种植风险。随机生成 10 组数据作为训练集，以 10 种情况下平均利润最高为目标，得到最优种植方案。再随机生成 50 组数据作为测试集，计算各情况下选用该方案得到的利润，对 50 组利润计算 CV 值、最大回撤值，从而对种植方案的抗风险能力进行评估。

2.3 问题三分析

对于问题三，我们在问题二的基础上充分考虑输入量之间的相关关系。我们引入不同作物之间的替代、互补效应，先计算各作物价格之间的 Spearman 相关系数，利用需求价格弹性和交叉价格弹性原理建立需求量与价格之间的关系。分析附件数据，我们发现作物售价与种植成本之间存在较强的线性相关性，即能通过确定各类作物种植成本来确定各类作物售价、销售量。接下来，受投资学领域单因子模型启发，我们将成本的变动分解为共同波动和个体波动，前者受宏观环境影响，后者则取决于特定作物的表现，两个因子均依据各自统计学规律进行随机模拟。我们以上述方法随机生成输入量，完善已有的最优种植方案算法，将得到的方案与问题二得到的方案进行比较，看是否更加符合现实逻辑因而能够获得更高的平均利润。

总而言之，问题一、问题二和问题三都属于**优化问题**，它们在种植特性和约束条件（如重茬限制、豆类要求等）具有一致性，优化目标也较为相似，但由于输入量不同，三者模型之间仍然存在显著差异。问题一的输入量是**固定的**，问题二则在限制范围内**完全随机变化**，而问题三的输入量则符合**一定的实际规律**，且输入量间存在**相互作用**。因

此，尽管它们在算法结构上有相似之处，输入量的不同导致了模型的具体表现和优化策略各异。

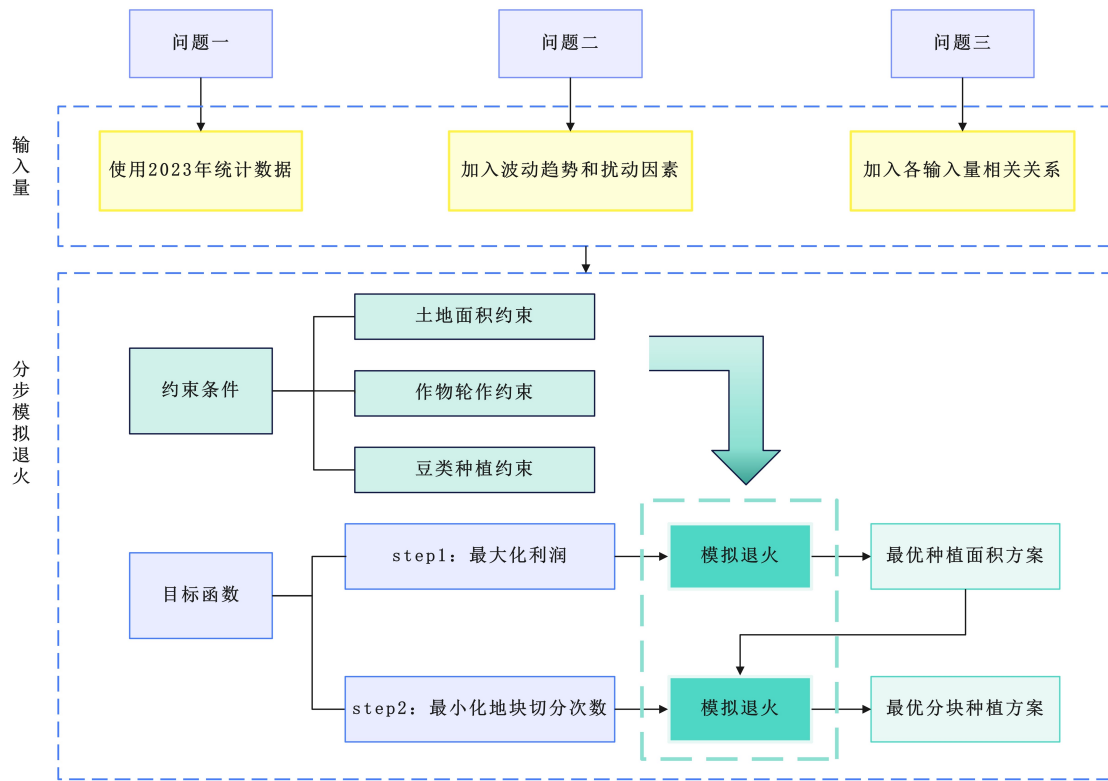


图1 总体思路图

三、模型假设与符号说明

3.1 模型假设

为简化问题，本文做出以下假设：

- **假设 1:** 针对各类作物的单价波动区间，我们均采用区间平均值估算销售收入。
- **假设 2:** 对于问题 1 和问题 2，我们假设 2023 作为基年实现了产销平衡，即销售量与产量相等，从而获得问题 1 的预期销售量和问题 2 的销售量浮动基准。
- **假设 3:** 忽略不同农作物合种可能带来的相互影响，如产量和质量的波动。

3.2 符号说明

符号	说明	单位
i	耕地类型编号	—
t	年份编号	—
j	作物编号	—
ϵ	某类型作物随机波动	—
k	波动相关系数	—
x	种植作物面积	亩
y	单位产量	斤/亩
Q	产量	斤
D	需求量	斤
P	销售价格	元/斤
C	总种植成本	元
c	每亩种植成本	元/斤
R	利润	元/亩
A	某类型耕地总面积	亩

四、数据侧写与预处理

4.1 数据侧写

本团队分析了该乡村现有各类型耕地分布情况，发现：梯田占比 51.0%，其次为平旱地，占比 30.1%。计算 2023 年各类型耕地带来的利润，梯田占比 44.8%。值得关注的是普通大棚耕地面积仅占 0.8%, 却带来了占比 8.5% 的利润; 山坡地面积占 8.9%，带来利润占比 13.0%。分析各类作物产量情况，蔬菜占比 43.1%，带来收入仅占 8.8%，粮食带来利润占比高达 83.5%。

2023 年统计数据显示，同种作物在不同耕地上每亩产量不同，粮食作物产量在平旱地、梯田、山坡地依次递减，种植成本保持不变。对于同一种蔬菜，智慧大棚相较于普通大棚产出少而单价高，产量集中在第二季。除此之外，在不同耕地上种植得到的同种作物售价相同。蔬菜种植成本于智慧大棚、普通大棚、水浇地依次递减。

4.2 数据预处理

首先，附件一、附件二的部分作物名称单元格内含有空格，为避免后续代码运行时出现误导，团队将空格删除，使所有作物名称格式统一。团队对每种作物的销售单价取

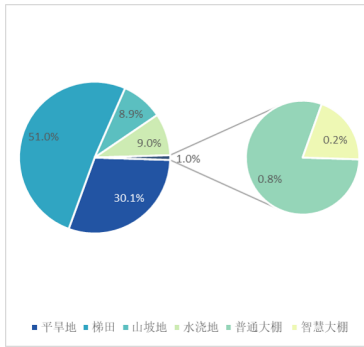


图2 各类耕地面积占比

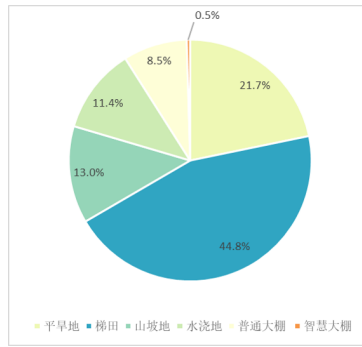


图3 各类耕地收入占比

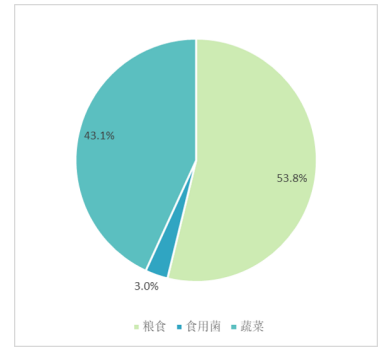


图4 三大类作物产量

上下限区间的中位数作为统一参考。这一做法既避免了极端价格波动对整体计算带来的偏差，更好地反映市场的平均价格水平，又可以简化计算。

其次，针对 2023 年各类农作物的种植数据，团队构建了一个二维矩阵。一维为所有作物的编号，每个编号对应一个具体的作物，另一维则包括每种作物的产量、销售单价及作物种类等信息。该矩阵的构建不仅能够整合所有数据，还为后续分析提供了直观的结构化数据支持。

再次，为了方便后续在模型中施加不同的条件约束，团队特别设置了五组布尔变量。这五个变量分别用于判断某一作物是否属于粮食、蔬菜、食用菌、豆类或水稻。布尔变量的设定使得我们能够根据不同作物类别，灵活应用不同的策略和约束条件，以提高决策的精准性。

并且，为了计算各类作物 2023 年的总产量，团队采用了将每类耕地的面积乘以对应作物亩产量的方法。随后，我们根据作物种类对产量进行加总，得出各类作物的总产量。该总产量作为基准需求量，将用于后续问题 1 和问题 2 中的计算分析。对于一些特殊处理情况，团队在问题求解过程中做了详细说明，以确保模型的完整性与准确性。

最后，我们将各类耕地类型的所有地块面积进行汇总，得到每种耕地类型的总面积，具体如表所示：

耕地类型	平旱地	梯田	山坡地	水浇地	普通大棚	智慧大棚
总面积/亩	365	619	108	109	9.6	2.4

表1 各耕地类型总面积

五、问题一的模型的建立和求解

多背包问题为问题一原型。该问题要求将一组物品分配到多个容量有限的背包中，以最大化物品的总价值。每个物品有固定的重量和价值，每个背包有容量限制。问题目标是在不超出背包容量的限制条件下，合理选择物品使得总价值最大化。多背包问题属

于 NP 难题，求解过程中需要在庞大的解空间中找到一个使得总价值最大的物品分配方案，模拟退火在这种情况下具有很大的优势。我们将问题拆分细化，采取**先不对耕地类型内部进行分块，确定各土地类型种植各类作物最优种植面积方案，再以此为限制条件，确定土地切分量最少的分块种植方案，分步解决问题**。分步运用模拟退火算法，相较于传统模拟退火可以减少每次的计算量，更灵活地处理约束条件，减小局部最优陷阱的风险。

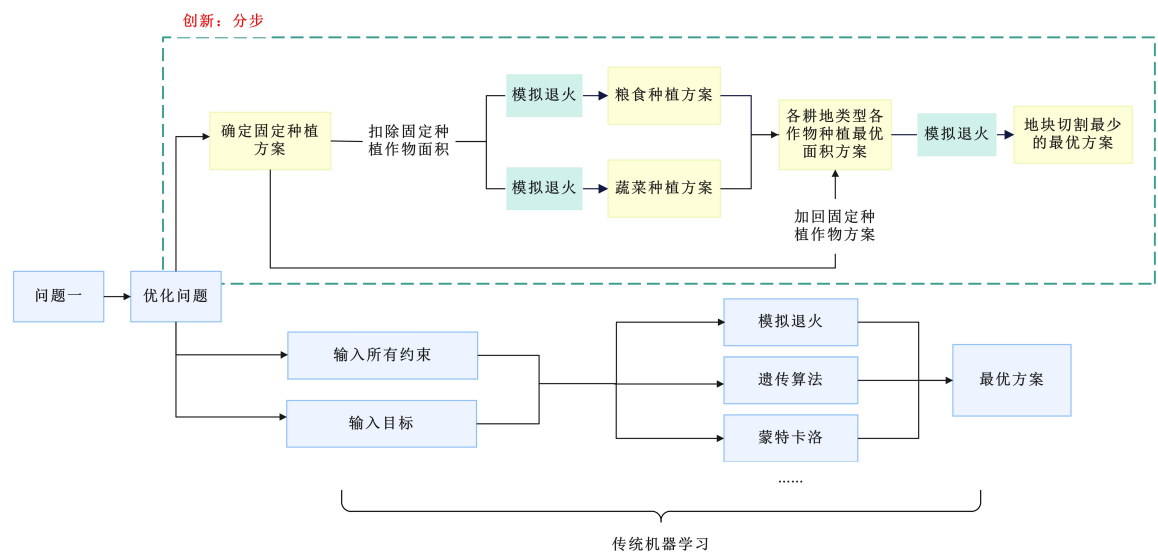


图 5 问题一流程图

5.1 模型建立

5.1.1 实现按耕地类型分类各作物种植面积最优化

Step1: 固定种植

由附件 1 中的说明信息分析可得下表，前两列作物有多种种植方案，后四列种植方案则较为确定。**(1) 水稻**只能在水浇地上种植，并且每年仅限单季种植。水浇地可以选择单季种植水稻，也可以选择两季种植蔬菜，且种植水稻所得利润明显低于种植蔬菜（即使蔬菜半价销售仍然高于正常价水稻），但 2023 年仍然选择种植水稻，我们认为存在政策规定等刚性需求，即水稻种植面积需满足相对固定的需求。**(2) 大白菜、白萝卜、红萝卜**这三类作物仅能在水浇地的第二季种植。由于水浇地种植水稻面积固定，种植蔬菜面积也因此确定。**(3) 食用菌**只能在第二季普通大棚中进行栽培，普通大棚第二季也只能进行食用菌种植，为达到最大化利润目的，食用菌种植面积能够确定。**(4) 第二季蔬菜（除大白菜、白萝卜、红萝卜）**其产量少单价高，较符合反季蔬菜特征，只能在第二季智慧大棚中进行栽培，其种植方案可直接由需求决定。问题一假定各种农作物未来的预期销售量、种植成本、亩产量和销售价格相对于 2023 年保持稳定，则相应耕地上可

以确定固定种植的作物面积。

耕地类型	单季粮食（除水稻）	第一季蔬菜	第二季蔬菜	水稻	三类冬季蔬菜	食用菌
平旱地	✓		✓			
水浇地		✓		✓	✓	
智慧大棚		✓				
普通大棚		✓				✓
梯田	✓					
山坡地	✓					

表 2 各种作物在不同耕地上的种植情况

上述四类作物对耕地有特殊使用要求，因此其土地占用变化空间小。我们先固定好四类作物的耕地占用。在分配耕地给其余作物前，先扣除这些作物所需的土地，能够简化后续其他作物的种植规划，同时确保不同种植需求之间的协调性与合理性。

去除固定种植后，如下表所示：

土地类型	平旱地	梯田	山坡地	水浇地	普通大棚	智慧大棚
每年第一季剩余面积/亩	365	619	108	67	9.6	2.4
每年第二季剩余面积/亩	——	——	——	0	0	0

表 3 去除固定种植作物后剩余耕地

Step2: 确定其他作物最优种植面积方案

确定固定种植作物面积后，单季粮食（除水稻）、第一季蔬菜需选用优化模型确定最佳种植方案，实现农作物种植的总利润最大化的目标。由于两类作物种植耕地类型并不交叉，我们分别使用模拟退火独立求解。

(1) 决策变量

将平旱地、梯田、山坡地、水浇地第一季、普通大棚第一季、智慧大棚第一季分别编号 $i=1,2,\dots,7$ 。令 $x_{i,j,t}$ 表示在第 i 种耕地类型、在第 t 年种植作物 j 的面积，决策变量为每种耕地类型、每年、每种作物的种植面积。

(2) 目标函数

我们希望最大化未来 7 年的种植总利润。在 i. 和 ii. 情况下，单季粮食（除水稻）、第一季蔬菜目标函数分别表示为：

$$Q_{\text{under}} = \min \left(\sum_{i=1}^3 Q_{i,j,t}, D_{j,t} \right), Q_{\text{over}} = \max \left(\sum_{i=1}^3 Q_{i,j,t} - D_{j,t}, 0 \right)$$

$$\max R_1 = \begin{cases} \sum_{j=1}^{15} \sum_{t=1}^7 (Q_{\text{under}} \times P_{j,t} - C_{j,t}) & \text{i.} \\ \sum_{j=1}^{15} \sum_{t=1}^7 (Q_{\text{under}} \times P_{j,t} - C_{j,t} + Q_{\text{over}} \times \frac{1}{2} P_{j,t}) & \text{ii.} \end{cases}$$

$$Q_{\text{under}} = \min(\sum_{i=4}^6 Q_{i,j,t}, D_{j,t}), Q_{\text{over}} = \max\left(\sum_{i=4}^6 Q_{i,j,t} - D_{j,t}, 0\right)$$

$$\max R_2 = \begin{cases} \sum_{j=17}^{34} \sum_{t=1}^7 (Q_{\text{under}} \times P_{j,t} - C_{j,t}) & \text{i.} \\ \sum_{j=17}^{34} \sum_{t=1}^7 (Q_{\text{under}} \times P_{j,t} - C_{j,t} + Q_{\text{over}} \times \frac{1}{2} P_{j,t}) & \text{ii.} \end{cases}$$

其中：

- $Q_{i,j,t} = x_{i,j,t} \times y_{i,j,t}$ 表示 j 作物在 i 类型耕地的产量；
- 在 R_1 目标式中, $C_{j,t} = \sum_{i=1}^3 (x_{i,j,t} \times c_{i,j,t})$; 在 R_2 目标式中, $C_{j,t} = \sum_{i=4}^6 (x_{i,j,t} \times c_{i,j,t})$, 其中 $C_{j,t}$ 为第 j 种作物在第 t 年的种植成本。
- $D_{j,t}$ 为每种作物的年需求量, 由 2023 年统计数据确定；
- $P_{j,t}$ 为每种作物的市场价格；

(3) 约束条件

a. 土地面积约束：每种耕地类型每年种植各类作物面积之和不能超过该类型耕地总面积，即：

$$\sum_{j=N}^M x_{i,j,t} \leq A_i, \quad \forall i, t$$

其中 A_i 是第 i 种耕地类型的总面积, 如表6所示, M 、 N 分别为该作物类型条目下作物编号的最大值和最小值。

b. 作物轮作约束：每种作物种植面积总和不超过该作物可种面积（后一年可种面积 = 总面积耕地 - 前一年某类型种植面积）。具体约束为：

$$\sum_{j=1}^M x_{i,j,t} \leq A_i - \sum_{j=1}^M x_{i,j,t-1}, \quad \forall i, t$$

c. 豆类作物种植约束：为满足后续分块确定种植方案时每块地在三年内至少要种植一次豆类作物，在本次计算中保证三年内豆类种植面积将该类型耕地总面积覆盖一次。即：

$$\sum_{t=1}^{t+1} \sum_{j \in \text{豆类}} x_{i,j,t} \geq A_i, \quad \forall i$$

5.1.2 确定最优分块种植方案

1. 决策变量

将各个地块分别编号, $x_{k,j,t}$ 表示在第 k 种耕地类型、在第 t 年种植作物 j 的面积, 决策变量为每种耕地类型、每年、每种作物的种植面积。

2. 目标函数

为了方便耕作和田间管理，每种作物在每个季节的种植区域应集中。我们的目标是 minimized 每块地切分次数，即最小化所有地块种植作物种类之和，该指标记作 Z ，我们希望通过调整地块的种植分配，达到避免种植作物区域过度分散的目的：

$$\min Z = \sum_{k=1}^L \sum_{t=1}^T \sum_{j=1}^C \mathbb{I}(x_{k,j,t} > 0)$$

- L 表示地块总数；
- T 表示年份的总数；
- C 表示作物的总种类数；
- $\mathbb{I}(x_{k,j,t} > 0)$ 是指示函数，如果该地块在该年份分配给该作物的面积大于 0，则返回 1，否则返回 0。

3. 约束条件

(1) 地块面积约束：每个地块 k 在每一年 t 的总种植面积不能超过该地块的可用面积 A_k ，即：

$$\sum_{j=1}^C x_{k,j,t} \leq A_k \quad \forall k \in \{1, \dots, L\}, \forall t \in \{1, \dots, T\}$$

(2) 避免连续重茬种植约束：为了避免连续两年种植同样的作物，要求对于每个地块 k ，同一作物 j 在连续两年的种植面积总和不超过地块面积：

$$x_{k,j,t} + x_{k,j,t-1} \leq A_k \quad \forall k \in \{1, \dots, L\}, \forall j \in \{1, \dots, C\}, t \geq 1$$

(3) 豆类作物种植约束：豆类作物的三年轮作约束要求在三年内分配给豆类作物的总面积必须满足一定条件。记豆类作物集为 B ，则：

$$\sum_{j \in \text{豆类}} \sum_{s=t-2}^t x_{k,b,s} \geq A_k \quad \forall k \in \{1, \dots, L\}, t \geq 2$$

5.2 模型求解

在确定粮食、第一季蔬菜（大白菜、白萝卜、红萝卜除外）最优种植面积方案以及确定分配地块方案时，我们使用模拟退火算法，在各类约束条件下有效搜索到全局最优解。模拟退火算法的核心思想是通过逐步降低温度 T ，在解空间中随机搜索最优解 [1]。设当前解为 x ，新解为 x' ，如果 $f(x') > f(x)$ ，则接受新解；如果 $f(x') \leq f(x)$ ，则以概率 $P = e^{\frac{f(x') - f(x)}{T}}$ 接受新解，其中 T 随着迭代逐渐减小。初期 T 较大，允许接受较多劣解，避免陷入局部最优；随着 T 的降低，算法趋向只接受较优解，最终收敛到全局最优解。

Step1: 初始化解与参数 设定初始温度 T_0 和冷却速率 α 。随机生成初始决策变量，并确保满足各项约束条件。

Step2: 计算初始利润 计算当前方案目标函数值，记录为初始解。

Step3: 随机生成新解 在每轮模拟退火中，随机调整决策变量，并重新检查满足各项约束条件。

Step4: 接受或拒绝新解 如果新方案得到目标函数值优于当前方案，则接受新解；否则根据模拟退火的概率公式，可能接受劣解以避免局部最优。

Step5: 降低温度并迭代 按照冷却率逐步降低温度，重复生成新解，直到温度降至设定的最低值 $T_{\min}$ 或达到最大迭代次数。最终返回找到的最佳决策变量和目标函数值。

5.3 求解结果及其分析

我们利用模拟退火算法，分别求出题给两种情况下粮食作物和第一季蔬菜的最优种植面积方案，进行多次迭代。得到的结果如图所示：

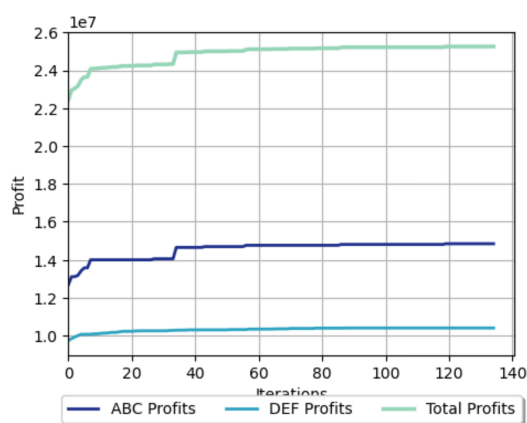


图 6 情况 i. 利润迭代情况

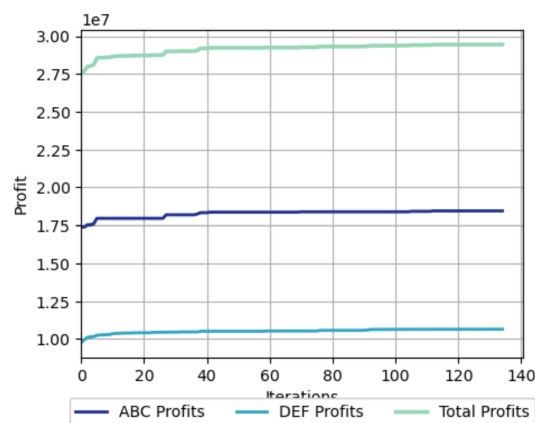


图 7 情况 ii. 利润迭代情况

其中 ABC 地块利润代表粮食作物利润，DEF 地块利润代表第一季蔬菜利润。总利润等于粮食作物利润、第一季蔬菜利润以及四类固定种植作物利润之和。情况 i、ii. 图表虽呈现出不同的增长特点，但总趋势一致。分析图表可知，前 40 次迭代中粮食作物利润明显上升，第一季蔬菜利润小幅度增长。最终总利润随迭代次数逐渐趋于稳定，说明算法成功收敛并找到了较为理想的作物种植方案。迭代得到的情况 i 和情况 ii 的总利润分别为：

- 情况 i 总利润 = 25866316
- 情况 ii 总利润 = 29156033

接下来，我们继续使用模拟退火算法，在已知各耕地类型的农作物种植需求的前提下，确定它们在不同地块上的分配方案。我们的目标是尽量减少地块的划分，使每种作

物在每个季节的种植区域保持相对集中。情况 i 和情况 ii 的结果如图所示，随迭代次数增加目标函数逐渐下降，且切线斜率绝对值增大。

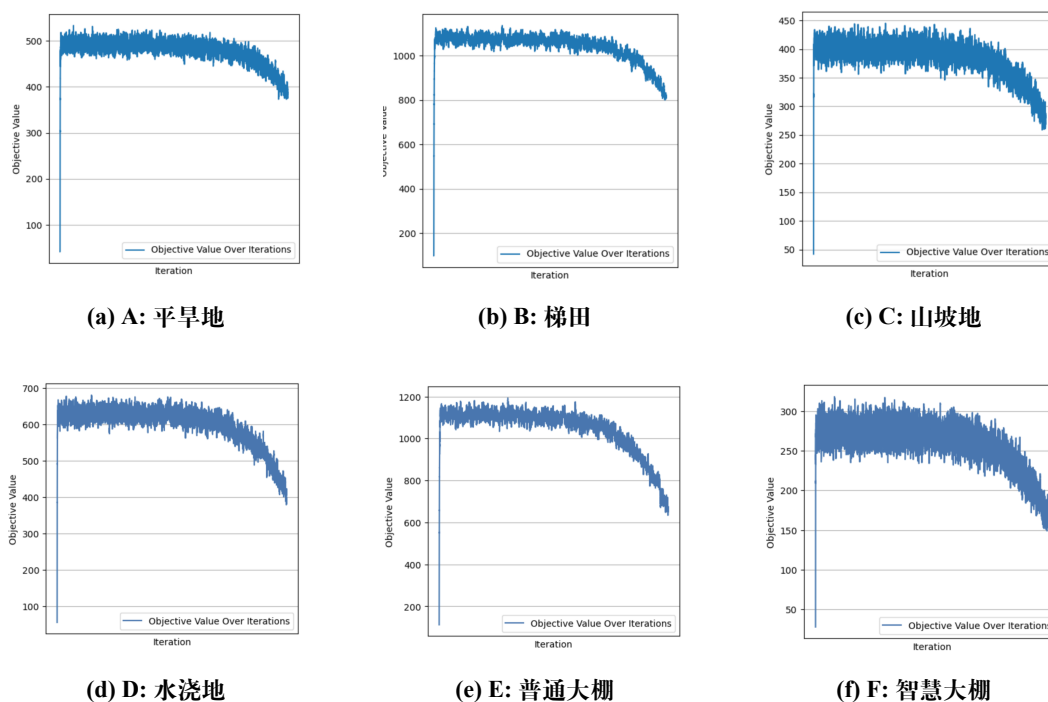


图 8 情况 i

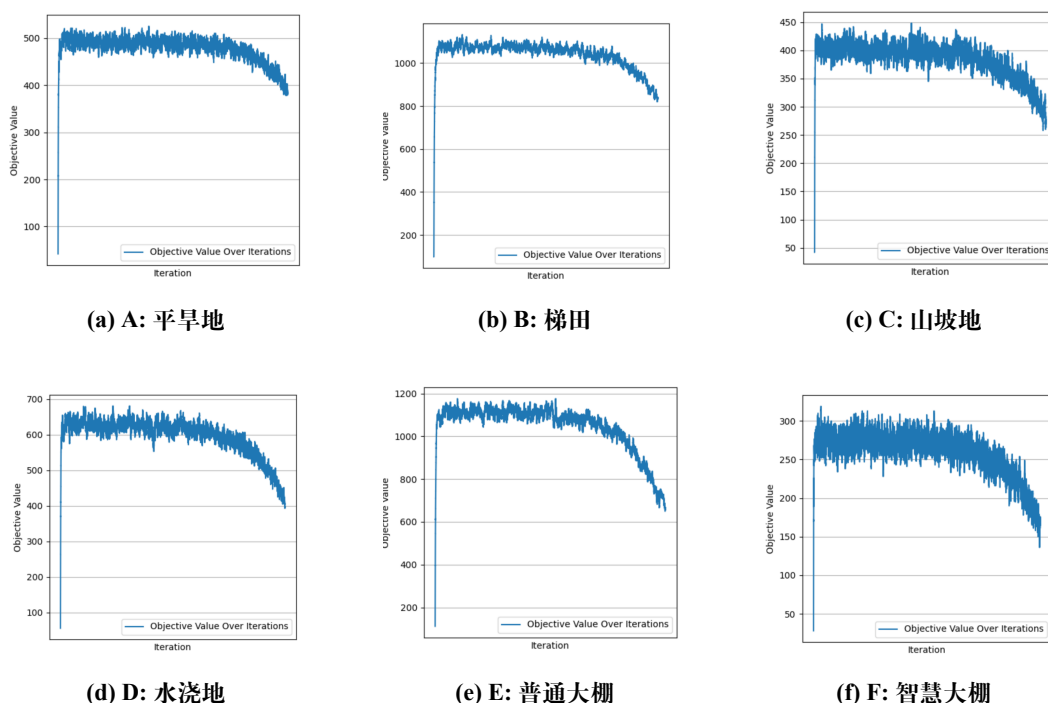


图 9 情况 ii

最后，根据各地块的农作物种植情况，填写表格。2024 年部分作物最优种植方案如下表所示，完整种植方案详见'result_1_1' 和'result_1_2'

地块名	黄豆	黑豆	小麦	玉米	谷子	高粱	水稻	豇豆	刀豆	土豆	西红柿	大白菜	白灵菇
A1	0.00	0.00	3.80	7.40	0.00	8.29	0.00	0.00	0.00	0.00	0.00	0.00	0.00
A2	0.00	7.82	4.29	0.00	0.00	8.70	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B1	1.69	2.67	2.90	4.67	0.00	2.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B2	1.34	0.00	0.00	0.00	19.16	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
C1	0.00	2.99	0.00	2.10	0.00	4.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00
C2	1.05	0.54	1.61	0.00	1.76	0.94	0.00	0.00	0.00	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.00	2.37	2.46	0.00	0.00	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.35	3.02	0.00	0.01	0.00	0.00	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.52	0.00	0.00
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.20	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	15.00	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.60
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.60
F1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.30	0.00	0.00
F2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

表 4 result_1_1: 2024 (部分)

地块名	黄豆	黑豆	小麦	玉米	谷子	高粱	水稻	豇豆	刀豆	土豆	西红柿	大白菜	白灵菇
A1	0.00	6.39	0.00	9.52	16.83	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
A2	3.95	0.00	1.05	1.67	7.82	0.13	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B1	0.00	0.00	3.20	0.00	3.00	7.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B2	8.57	6.15	6.06	0.00	8.29	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
C1	0.22	0.00	0.00	0.00	0.00	2.19	0.00	0.00	0.00	0.00	0.00	0.00	0.00
C2	0.00	0.03	5.56	0.00	0.00	0.49	0.00	0.00	0.00	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	3.39	2.08	1.56	3.11	0.00	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2.77	2.39	0.00	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.39	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	15.00	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	15.00	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.60
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.60
F1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.27	0.00	0.00
F2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00

表 5 result_1_2: 2024 (部分)

六、问题二的模型的建立和求解

问题二在问题一基础上加入各项数据的变化趋势和扰动因素。结合农作物的预期销售量、亩产量、种植成本及其销售价格的不确定性，以及可能存在的种植风险，为该乡村制定 2024 至 2030 年期间的最优农作物种植规划。

6.1 模型建立

6.1.1 加入随机性

根据经验得到各统计数据变化趋势信息，我们将这些信息转化为数学语言并加入随机扰动，随机扰动使用 $N(0, \sigma^2)$ 表示。

(1) 销售量（需求量）

对于小麦和玉米，预期需求量年均增长 5%~10%，加入随机扰动：

$$D_{\text{wheat or corn},t} = D_{\text{wheat or corn},2023} \cdot \prod_{i=2023}^{t-1} (1 + U(0.05, 0.10) + N(0, \sigma_D^2))$$

对于其他作物，需求量年波动 $\pm 5\%$ ：

$$D_{\text{other},t} = D_{\text{other},t-1} \cdot (1 + N(0, 0.05^2))$$

(2) 亩产量

农作物的亩产量受气候影响，每年波动 $\pm 10\%$ ：

$$y_{\text{crop},t} = y_{\text{crop},2023} \cdot (1 + N(0, 0.10^2))$$

(3) 种植成本

每亩种植成本每年增长 5%，加入随机扰动：

$$c_{\text{crop},t} = c_{\text{crop},2023} \cdot \prod_{i=2023}^{t-1} (1.05 + N(0, \sigma_c^2))$$

(4) 销售价格

对于粮食作物，价格保持稳定，加入少量随机扰动：

$$P_{\text{grain},t} = P_{\text{grain},2023} \cdot (1 + N(0, \sigma_P^2))$$

对于蔬菜类作物，价格年均增长 5%，加入随机扰动：

$$P_{\text{vegetable},t} = P_{\text{vegetable},2023} \cdot \prod_{i=2023}^{t-1} (1.05 + N(0, \sigma_P^2))$$

对于食用菌（除羊肚菌），价格每年下降 1%~5%，加入随机扰动：

$$P_{\text{mushroom},t} = P_{\text{mushroom},2023} \cdot \prod_{i=2023}^{t-1} (1 + U(-0.05, -0.01) + N(0, \sigma_P^2))$$

对于羊肚菌，价格每年下降 5%，加入随机扰动：

$$P_{\text{morchella},t} = P_{\text{morchella},2023} \cdot \prod_{i=2023}^{t-1} (0.95 + N(0, \sigma_P^2))$$

我们使用现实数据确定扰动方差，以价格扰动方差为例介绍计算方法：

$$\begin{cases} r_m = \ln \left(\frac{P_m}{P_{m-1}} \right) \\ \mu = \frac{1}{N} \sum_{m=1}^N r_m \\ \sigma^2 = \frac{1}{N-1} \sum_{m=1}^N (r_m - \mu)^2 \end{cases}$$

其中：

- r_m 表示第 t 期的对数收益率，
- P_m 为第 m 期的价格，
- P_{m-1} 为前一期的价格，
- μ 表示平均收益率，
- N 为总的交易天数，
- σ^2 表示收益率的方差（扰动方差），

通过计算，我们得出：

- 不同作物的成本扰动方差近似，为简化计算，我们认为 $\sigma_D^2 = 0.5^2$
- 不同作物的销量扰动方差、价格扰动方差存在较大差异，详见下表格：

	黄豆	红豆	土豆	西红柿	……	小麦	玉米	……	香菇
σ_D^2	——	——	——	——	……	0.005	0.010	……	——
σ_P^2	0.223	0.005	0.024	0.026	……	0.005	0.001	……	0.036

表 6 销量扰动方差、价格扰动方差汇总

6.2 模型求解

6.2.1 确认目标函数

通过引入随机性，我们在确保符合经验趋势的前提下对各输入量添加扰动因素，来模拟现实生活中潜在的种植风险。由于本题旨在求解具有一定抗风险能力的最优种植方案，我们选用在随机生成的不同情况下的平均利润最大为目标。我们随机生成 10 组输入量，分别编号 $q=1,2,3,\dots,10$ 。决策变量为 $x_{i,j,t}$ ，目标为使十种情况下平均利润最高，

约束条件设置逻辑与问题一相同。 R_1 、 R_2 分别为单季粮食（除水稻）、第一季蔬菜利润。

$$\begin{aligned}
Q_{\text{under}} &= \min \left(\sum_{i=1}^3 Q_{i,j,t,q}, D_{j,t,q} \right), Q_{\text{over}} = \max \left(\sum_{i=1}^3 Q_{i,j,t,q} - D_{j,t,q}, 0 \right) \\
R_{1,q} &= \sum_{j=1}^{15} \sum_{t=1}^7 \left(Q_{\text{under}} \times P_{j,t,q} - C_{j,t,q} + Q_{\text{over}} \times \frac{1}{2} P_{j,t,q} \right) \\
\max \bar{R}_1 &= \frac{1}{10} \sum_{q=1}^{10} R_{1,q} \\
Q_{\text{under}} &= \min \left(\sum_{i=4}^6 Q_{i,j,t,q}, D_{j,t,q} \right), Q_{\text{over}} = \max \left(\sum_{i=4}^6 Q_{i,j,t,q} - D_{j,t,q}, 0 \right) \\
R_{2,q} &= \sum_{j=17}^{34} \sum_{t=1}^7 \left(Q_{\text{under}} \times P_{j,t,q} - C_{j,t,q} + Q_{\text{over}} \times \frac{1}{2} P_{j,t,q} \right) \\
\max \bar{R}_2 &= \frac{1}{10} \sum_{q=1}^{10} R_{2,q}
\end{aligned}$$

其中：

- $Q_{i,j,t,q} = x_{i,j,t} \times y_q$ 表示在第 q 种情况下， j 作物在 i 类型耕地的产量；
- 在 R_1 目标式中， $C_{j,t,q} = \sum_{i=1}^3 (x_{i,j,t} \times c_{i,j,t,q})$ ；在 R_2 目标式中， $C_{j,t,q} = \sum_{i=4}^6 (x_{i,j,t} \times c_{i,j,t,q})$ ，其中 $C_{j,t,q}$ 为在第 q 种情况下，第 j 种作物在第 t 年的种植成本。
- $D_{j,t,q}$ 为在第 q 种情况下每种作物的年需求量；
- $P_{j,t,q}$ 为在第 q 种情况每种作物的市场价格。

以上为实现按耕地类型分类各作物种植面积最优化的目标函数，使用模拟退火算法现按耕地类型分类各作物种植面积最优化。在下一步骤中我们确定最优分块种植方案，与问题一方法一致，得到最终最优种植方案。

6.2.2 方案评估

在确定了最优种植方案之后，为了评估该方案在不同情况下的稳定性和抗风险能力，我们生成了 50 组随机输入数据作为测试集。这些数据涵盖了销售量、亩产量、种植成本和销售价格等变量，代表了不同的市场和生产环境。

每组测试数据都模拟了一种可能的实际情况，我们使用这些数据来测试最优种植方案的表现。具体来说，我们根据当前最优种植方案对每组新的输入数据进行计算，得出相应的利润。这一过程的目的是评估在不同情况下最优种植方案的表现和稳定性，从而了解其是否能在实际操作中保持预期的经济效益。

6.3 求解结果及其分析

通过模拟退火算法的迭代优化，我们得到了最优的种植方案，该方案在训练集中的平均利润最高，达到了 **23781943**。其对应的测试集的利润分布与统计学指标如下所示：

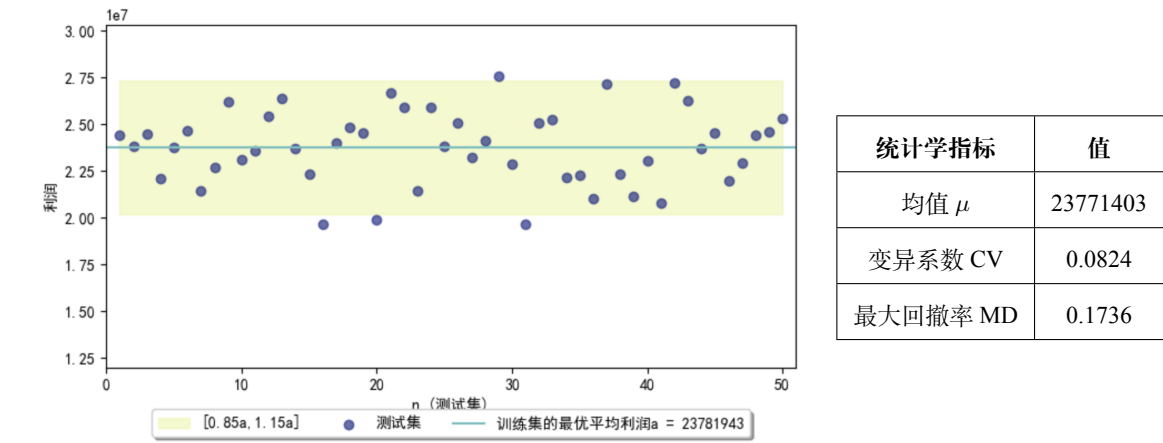


图 10 问题二测试集

其中：

变异系数 $CV = \frac{\text{标准差}(\sigma)}{\text{均值}(\mu)}$

最大回撤率 $MD = \frac{\text{训练集最优平均利润} - \text{测试集利润最小值}}{\text{训练集最优平均利润}}$

图中水平线为训练集最优平均利润 a ，绿色条带为以该数据为中心设置的参考范围 $[0.85a, 1.15a]$ 。图中测试集利润数据点大部分位于条带内，说明该种植方案在不同市场和生产环境下获得利润具有一定的稳健性和可靠性。

为评估方案抗风险能力，我们分别计算**变异系数**（CV）、**最大回撤率**（MD）。变异系数用来衡量利润相对于均值的波动程度，得到 $CV < 0.1$ ，显示该种植方案在不同情境下的收益风险较低。最大回撤率用来衡量种植方案在最不利情况下的最大可能损失，求得 $MD < 0.2$ ，显示该方案的风险相对可控。

以上分析可知，所得最优种植方案在盈利能力和抗风险能力方面表现良好，在不同市场和生产环境下表现较为稳健。

因而，该最优种植方案符合问题二的要求，我们将其填到 ‘result_2’ 中，部分数据如下表所示。

地块名	黄豆	黑豆	小麦	玉米	谷子	高粱	水稻	豇豆	刀豆	土豆	西红柿	大白菜	榆黄菇
A1	7.78	5.95	12.90	4.80	1.38	12.65	0.00	0.00	0.00	0.00	0.00	0.00	0.00
A2	0.00	3.16	9.65	2.02	0.00	4.55	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B1	3.44	0.00	10.12	2.70	6.84	11.07	0.00	0.00	0.00	0.00	0.00	0.00	0.00
B2	0.00	0.00	0.00	5.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
C1	3.70	0.00	0.00	0.83	0.10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

C2	0.00	4.55	0.28	2.07	0.00	2.61	0.00	0.00	0.00	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.67	0.00	0.00	0.00	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.96	0.91	0.00	0.00	0.00	0.00	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.00	0.00	0.53	0.00	0.00
F1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00	0.00	0.00
F2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.00	0.00	0.00	0.00
D1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	14.93	0.00
D2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	9.98	0.00
E1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.29
E2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.24
F1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.30	0.00	0.00
F2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

表 7 result_2: 2024 (部分)

七、问题三的模型的建立和求解

在问题三中，我们改进了问题二生产成本、价格和需求量的无规则扰动，通过经济学、投资学相关知识，建立生产成本、价格和需求量的内在联系，使这三种要素的波动更符合市场规律和内在逻辑。通过 Spearman 相关系数来确定作物之间的价格相关性，选取其中相关性显著的作物，构建替代关系和互补关系，得出各类农产品的需求量矩阵，让农产品的需求波动更符合市场规律。相互替代品和互补品是微观经济学中的重要概念。替代品是指当一种商品的价格上升时，消费者可以转向购买另一种具有类似功能的商品。互补品则是指两种商品需要一起使用，当其中一种商品的价格下降时，另一种商品的需求也会增加。

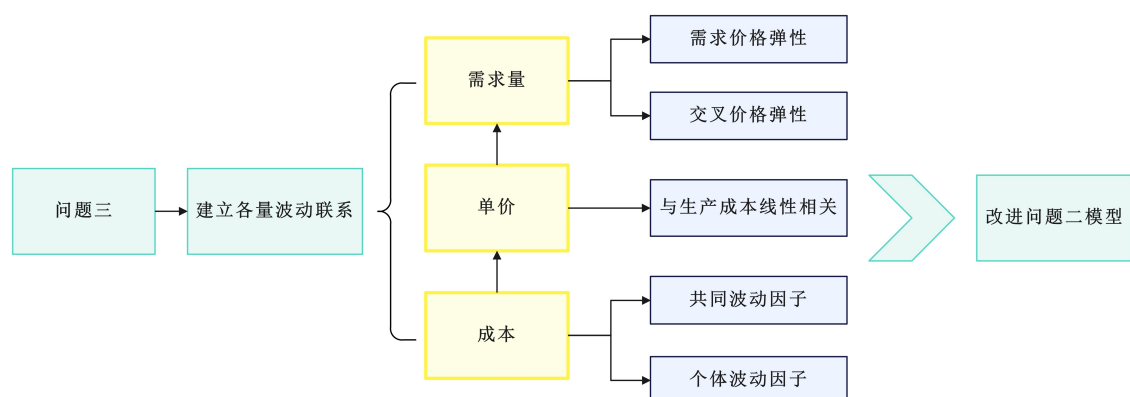


图 11 问题三流程图

7.1 模型建立

我们分析了自 2016 年 8 月到 2020 年 9 月的农产品价格统计表，将缺漏的部分用前后的平均数补齐，进行 Spearman 相关性分析，得到各类作物价格的相关性系数。

7.1.1 需求量的确定

在经济学中，正常商品是指当消费者收入增加时，需求量也随之增加的商品。对于正常商品，价格与需求量通常呈现反向关系，这就是所谓的需求定律。具体来说，当正常商品的价格上升时，消费者的购买成本增加，导致其需求量下降；反之，当价格下降时，消费者能够购买更多的该商品，因此需求量会上升。这种价格和需求量的反向关系是市场经济中普遍存在的现象。

农产品属于正常商品范畴，符合价格和需求量的负相关规律 [2]。结合我们的相关性系数可以得出，A 和 B 的价格相关系数为正值，说明 A 和 B 价格正相关，则 A 的价格和 B 的需求量成负相关，即 A 和 B 为互补品。反之，A 和 B 的价格相关系数为负值，则 A 的价格和 B 的需求量成正相关，即 A 和 B 为替代品。在问题 2 中，需求量的变动是根据我们的所设立的条件独立随机产生的；为更好地体现出实际农产品市场价格-需求规律，我们在此引入经济学中的弹性概念，帮助我们刻画作物的需求量，反应农产品间的内在联系。

在经济学中，弹性是指某一经济变量对另一变量变化的敏感程度，特别是价格变化对需求量或供给量的影响程度。弹性用来衡量价格变动对需求量或供给量变化的响应程度，是市场分析中的一个重要概念。一个商品的需求量变动受自身价格变动以及替代品、互补品的价格变动影响。需求价格弹性和交叉价格弹性是经济学中分别描述这两种关系的工具。

需求价格弹性衡量了某一商品的需求量对其自身价格变化的敏感程度。需求价格弹性可以用于分析价格变动对需求量的影响，公式如下：

$$E_d = \frac{\Delta Q}{\Delta P} \times \frac{P}{Q}$$

其中：

- E_d ：需求价格弹性；
- ΔQ ：商品需求量的变化；
- ΔP ：商品价格的变化；

交叉价格弹性用来衡量一种商品的需求量对另一种商品价格变化的敏感性，尤其用于分析替代品和互补品之间的关系。交叉价格弹性的公式为：

$$E_{xy} = \frac{\frac{\Delta Q_x}{Q_x}}{\frac{\Delta P_y}{P_y}} = \frac{\Delta Q_x}{\Delta P_y} \times \frac{P_y}{Q_x}$$

其中：

- E_{xy} ：商品 x 对商品 y 价格的交叉价格弹性；
- ΔQ_x ：商品 x 的需求量变化；
- Q_x ：商品 x 的初始需求量；
- ΔP_y ：商品 y 的价格变化；
- P_y ：商品 y 的初始价格。

综合以上两种弹性，我们选取作物自身价格变动和所有 Spearman 相关性系数绝对值高于 0.5 作物价格变动纳入需求量变动测定范围，根据以上两类弹性公式进行变式和加总，提出我们的需求量变动计算公式：

$$\frac{\Delta Q_0}{Q_0} = -\frac{\epsilon}{2} \frac{\Delta P_0}{P_0} - \frac{\epsilon}{2} \sum \left(\frac{k_i}{\sum k_i} \frac{\Delta P_i}{P_i} \right)$$

其中：

- ΔQ_0 ：所求商品的需求量变化；
- ΔP_0 ：所求商品的价格变化；
- k_i ：作物 i 对所求作物价格的 Spearman 相关系数， $i>1$ 为所有满足 Spearman 系数绝对值高于 0.5 作物

7.1.2 单价的确定

作物的单价受生产成本、运输成本、劳务成本和税收等因素影响 [3]，对于不同种类作物，最大影响因素为生产成本。

作物的成本受众多因素影响，如生产成本、技术进步、自然灾害、国际形势、政府补贴需要注意的是，我们利用每亩成本和亩产量计算出每斤成本，只有每斤成本和单价有可比性。经过回归拟合计算，我们发现将生产成本进行单变量拟合所得 Pearson 相关系数为 0.94，拟合程度高。为简化计算，提高算法效率，我们将作物单价设置为与生产成本有关的线性关系。因此我们可以得出单价计算公式：

$$\frac{\Delta P_i}{P_i} = k \frac{\Delta C_i}{C_i} + \epsilon_i$$

其中：

- ΔC_i ：商品 i 的生产成本变化；
- k ：通过拟合得到的 Pearson 相关系数，通过附件数据所得 $k=0.94$

7.1.3 生产成本的确定

为探究单位生产成本的决定公式，我们在此引入投资学中的单因子模型进行类比，便于我们建立单位生产成本模型。

单因子模型是投资学中用于解释证券收益波动的一个基本模型 [4]。该模型假设证券的收益主要受到多个个共同的风险因子影响，因此称为“单因子”模型。这个模型简化了对资产价格波动的分析，帮助投资者理解不同资产的共同风险来源。

单因子模型的基本形式为：

$$R_i = \alpha_i + \beta_i F + \epsilon_i$$

其中：

- R_i ：第 i 个资产的实际收益率；
- α_i ：资产 i 的超额收益；
- β_i ：资产 i 对风险因子的敏感性，即贝塔系数；
- F ：单一风险因子，通常表示市场组合的收益率或宏观经济变量；
- ϵ_i ：资产 i 的特有风险。

同样地，单位生产成本会受生产要素成本、气候和栽培方式等影响。我们可以采用与单因子模型相似的思路，将其抽象为个体波动因子和共同波动因子。共同波动因子，也可以通俗理解为“丰年”、“欠年”，代表宏观共同因素对单位生产成本的影响。如果当年遇到极端干旱，用水成本升高会导致所有作物每斤生产成本升高。个体波动因子则为微观因素对某一种作物的影响，如土地肥力缺失或疾病虫害会使某几种作物亩产量减少，每斤成本提高。综合上述两个因子，我们类比单因子模型提出单位生产成本变动的决定公式：

$$\frac{\Delta C_{i,t}}{C_{i,t-1}} = F_t + \epsilon_{i,t}$$

其中：

- $\Delta C_{i,t}$ ：第 i 种作物第 t 年的单位生产成本变化；
- $C_{i,t-1}$ ：第 i 种作物第 $t-1$ 年的单位生产成本；
- F_t ：第 t 年的共同波动因子；
- $\epsilon_{i,t}$ ：第 i 种作物第 t 年的个体波动因子。

7.2 模型求解

Step1：求解生产成本、单价和需求量

通过在模型建立中的分析，我们可以明确三者求解的先后关系，分别为生产成本、单价和需求量。生产成本的变量完全由随机扰动构成，可以率先确定。通过程序设定好随机扰动，可以得到各类作物关于生产成本的数组。然后根据我们求出的 Pearson 系数，能够用每种作物的生产成本求解出作物单价。

在构建 Spearman 相关性矩阵时，由于我们能够获取的数据有限，我们找到的作物历史价格数据没能完全覆盖题目中的所有作物。因此，我们将建模中的理想公式划分为以下两种情况：

(1) 当作物的 Spearman 相关系数存在绝对值大于 0.5 时：

$$\frac{\Delta Q_0}{Q_0} = -\frac{\epsilon}{2} \frac{\Delta P_0}{P_0} - \frac{\epsilon}{2} \sum \left(\frac{k_i}{\sum k_i} \frac{\Delta P_i}{P_i} \right)$$

(2) 当缺少作物的相关性系数或无显著的相关作物时：

$$\frac{\Delta Q_0}{Q_0} = -\epsilon \frac{\Delta P_0}{P_0}$$

以上三个因素的决定公式都包含个体波动因子 ϵ ，一般情况下与系统性无关的波动，我们认为期望为 0。因此，我们采用了最常见的正态分布来描述个体波动因子的分布，即 $\epsilon \sim N(0, \sigma^2)$ 。而成本决定公式中的 F_t 是由宏观因素决定的影响因子，从宏观经济规律来看，随着通货膨胀进程的持续，生产成本长期有上升的趋势，所以 F_t 服从的正态分布均值可以保守估计为 0.01。得到 2024 年-2030 年各类作物分别关于生产成本、单价和需求量的数组后，我们就为求解最优化做好了数据上的准备，进而调整问题 1、2 的程序以对接我们建立的反映市场规律的生产成本、单价和需求量三个数组，进行求解。

Step2: 改进、利用问题 2 所用模型求解最优种植面积方案

在问题 2 中，题目将生产成本、价格、需求量的变化趋势固定，作为已知值放入我们建立的模型，从而规划种植面积和耕地划分等因素。在问题 3 中，我们新建立的模型的创新点在于让成本、价格和需求量的变化建立了三者相互的联系，以及作物之间的联系，让三个要素的变化更加符合市场规律，符合逻辑，从而增加模型的真实性和拟合效果。

而当我们通过问题 3 新建立的模型确定好这三个要素的具体数值后，此时的问题求解就成为了与问题 2 要求相同的情况，其与问题 2 的已知量相同，求最优种植策略方法也就与问题 2 相通。

7.3 求解结果及其分析

7.3.1 农作物替代性、互补性

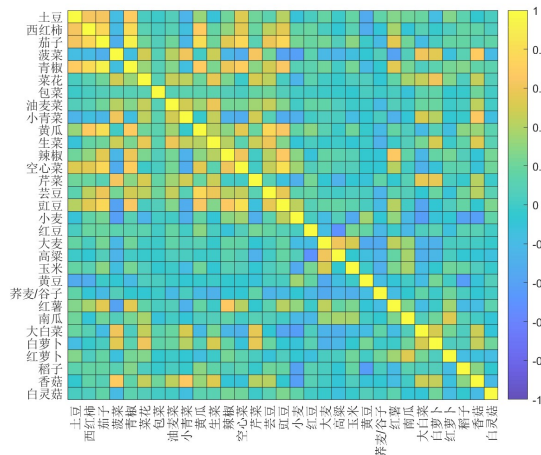


图 12 Spearman 相关系数热图

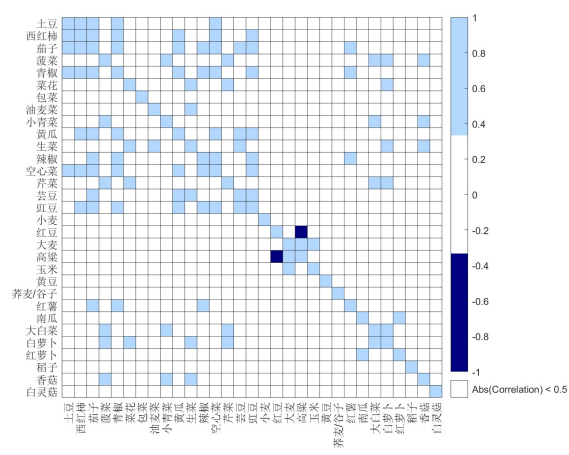
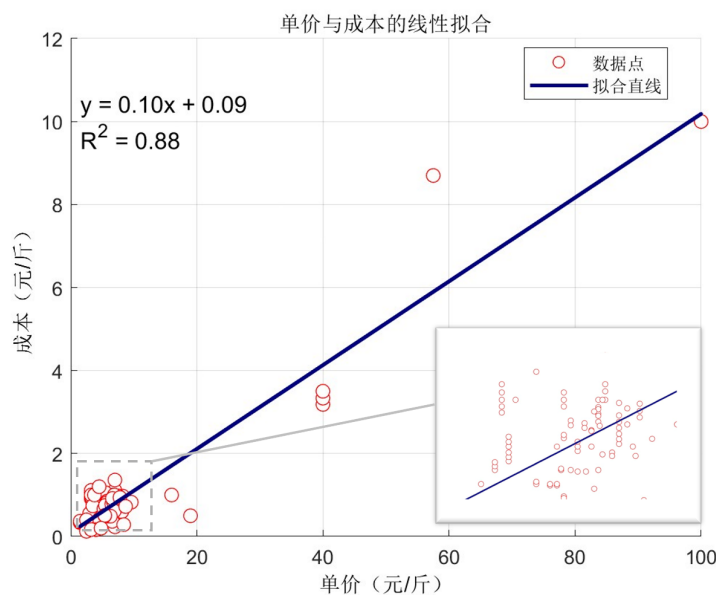


图 13 Spearman 相关系数分类筛选图

我们将查找到的现实数据进行统计学分析，计算各个作物价格之间的 Spearman 系数，如上图所示。由图可明显看出，多对蔬菜作物价格之间有较强相关性，而粮食类作物之间、粮食作物与蔬菜作物之间较少表现出强相关性。我们设置 0.5 为相关性系数门槛，超过这一门槛的纳入替代效应和互补效应的考察范围，根据系数的大小来确定影响的权重。



统计学指标	值
Pearson 相关系数 r	0.93969
p 值	9.8499×10^{-51}

图 14 单价与成本的线性关系

上方图标能够清晰看出单价与生产成本有明显的线性关系。通过简单的线性回归拟

合就能够充分体现两者的变动关系。因此为简化计算，提高算法效率，我们在此问题中将价格视为由生产成本单变量决定。

7.3.2 最优种植策略利润

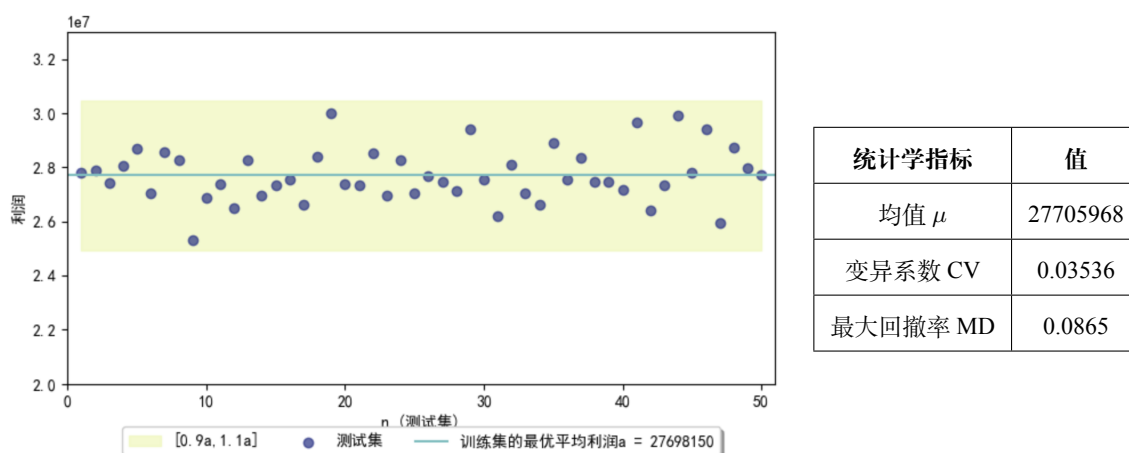


图 15 问题三测试集

图中水平线为训练集最优平均利润 a ，绿色条带为以该数据为中心设置的参考范围 $[0.9a, 1.1a]$ 。图中测试集利润数据点全部位于条带内，说明该种植方案在随机生成的不同市场和生产环境下获得的利润稳健性较高。我们分别计算**变异系数**（CV）、**最大回撤率**（MD）来评估最优方案的抗风险能力。变异系数用来衡量利润相对于均值的波动程度，最大回撤率用来衡量种植方案在最不利情况下的最大可能损失，求得 $CV < 0.04, MD < 0.09$ ，显示该方案抗风险能力较强。

统计学指标	值
均 值 μ	23771403
变异系数 CV	0.0824
最大回撤率 MD	0.1736

问题二

统计学指标	值
均 值 μ	27705968
变异系数 CV	0.03536
最大回撤率 MD	0.0865

问题三

表 8 问题二与问题三测试集统计指标对比

通过对比问题 2 和问题 3 测试集的统计指标，我们能够清晰地看到，加入相关经济学逻辑后形成的“半随机”波动条件下，所产生的生产成本、单价和需求量在输入模型后，产生的平均总利润明显高于完全随机参数下的平均总利润。这表明，经济学逻辑的引入优化了生产条件，使得收益结果更为理想。

此外，“半随机”条件下的变异系数明显较低，这意味着总利润的波动性较小，说明在这种条件下的收益更为稳定。最大回撤的数值也显示出“半随机”条件下的最大可能损失更小，进一步验证了这一稳定性。因此，从变异系数和最大回撤这两个角度来看，半随机参数的使用提高了模型的鲁棒性，使得收益更加可预测。

这一结论进一步表明，优化模型参数的相关性，能够使模型在实际应用中表现得更为出色。因此，建立更准确的参数关联关系，能够作为模型优化的一个重要方向，进一步提高模型的预测能力与稳健性。

八、模型的评价

8.1 模型的优点

- 优点 1 我们根据作物的条件约束，将作物分配问题拆分为几个步骤先后进行。白菜萝卜、食用菌、水稻等作物种植要求十分特殊，在考虑其他作物分配问题前，先将其种植面积确定。随后又以利润最大化为目标，先确定各类作物在每一时间节点的种植面积，再根据种植面积求解出最优划分。这样的拆分求解能有效减少未知量，降低建模的难度，提高了团队建模的效率，实现算法简化。

- 优点 2 在考虑生产成本、产品价格和需求量的输入量时，我们引用经济学、投资学等相关学科专业知识和工具进行融合和类比，提高了模型的可信性、严谨性，也让读者更加易懂我们的模型思路。

- 优点 3 模型泛用性强。从问题 1 到问题 3，我们建立的模型的求解思路和所需参数类型始终一致而连贯——先分配面积再划分地块。在问题 1、2、3 中，我们对输入量进行了不同程度的处理，而模型的核心算法始终保持一致，并且在不同条件的参属下保持着良好的稳定性。

8.2 模型的缺点

- 缺点 1 由于时间有限，划分地块步骤中的模拟退火运行次数不够大，所求得地块划分最优方案可能偏离实际最佳方案较大。

- 缺点 2 在波动参数的设置上，我们缺乏更加严谨的资料查阅和论证，让我们建立的市场内在联系与实际的市场联系会有差距。在搜寻我国农作物价格资料时，未能全面覆盖题目所给的所有农作物，分析时选取的时间跨度也受限。

参考文献

[1] 庞峰. 模拟退火算法的原理及算法在优化问题上的应用[D]. [出版地不详]: 吉林大学, 2006.

- [2] 俞宝. 考虑替代效应的农产品供应链定价策略研究[D]. [出版地不详]: 中国科学技术大学, 2017.
- [3] 严太华, 战勇. 农产品定价权研究——基于中间利润分成模型的分析[J/OL]. 财经研究, 2005(10):116-123. DOI: 10.16538/j.cnki.jfe.2005.10.012.
- [4] 马庆魁. 我国货币市场利率期限结构及其与宏观经济关联性研究[D]. [出版地不详]: 吉林大学, 2009.
- [5] 唐瑞, 郭健, 罗忠, 等. 鼠笼式弹性支承结构参数分步优化设计方法[J/OL]. 航空发动机, 2016, 42(02):38-43. DOI: 10.13477/j.cnki.aeroengine.2016.02.008.

附录 A 支撑材料列表

1.1 需要提交的结果

1. **result1_1.xlsx**: 问题一在情况 i 下的最优种植策略
2. **result1_2.xlsx**: 问题一在情况 ii 下的最优种植策略
3. **result2.xlsx**: 问题二的最优种植策略

1.2 代码

1. **q1_2_1.py**: 模拟退火代码, 问题一在情况 i 下各作物在不同耕地类型的最优策略
2. **q1_2_1.py**: 模拟退火代码, 问题一在情况 ii 下各作物在不同耕地类型的最优策略
3. **qn_2_最优地块分配.py**: 模拟退火代码, 各作物在不同地块的最优策略
4. **qn_3_填入result.py**: 将数据填入 result 表格中
5. **q2_0.py**: 问题二计算输入量代码
6. **q2_2.py**: 问题二模拟退火代码, 各作物在不同耕地类型的最优策略
7. **q3_0.py**: 问题三计算输入量代码
8. **q3_2.py**: 问题三模拟退火代码, 各作物在不同耕地类型的最优策略

1.3 其他佐证材料

1. **价格数据.xlsx**: 国家统计局等网站查找各类作物历史价格数据
2. **需求量数据.xlsx**: 国家统计局等网站查找各类作物需求量历史数据
3. **数据侧写与分析.xlsx**: 对附件一、附件二进行的数据分析

附录 B 重要代码

其他代码请见支撑材料文件夹

1. q1_2.py

```
1  -*- coding: utf-8 -*-
2
3  # 情况1
4  pip install openpyxl
5
6  import pandas as pd
7  import numpy as np
8  import random
9  import matplotlib.pyplot as plt
10
11 # 基础数据初始化
12 num_years = 7 # 2024——2030
13 # ABC: 粮食类
14 areas = {'A': 385, 'B': 619, 'C': 108} # ABC土地类型的面积
15 num_lands_ABC = 3 # ABC是粮食类
16 n1_A = 140 # A: 第0年豆类种植量 (单位: 亩)
17 n1_B = 199 # B: 第0年豆类种植量 (单位: 亩)
18 n1_C = 35 # C: 第0年豆类种植量 (单位: 亩)
19
20 # DEF: 蔬菜类
21 areas_DEF = {'D': 67, 'E': 9.6, 'F': 2.4} # DEF土地类型的面积
22 num_lands_DEF = 3 # DEF是蔬菜类地块
23 n1_D = 22 # D: 第0年豆类种植量 (单位: 亩)
24 n1_E = 3.6 # E: 第0年豆类种植量 (单位: 亩)
25 n1_F = 1.2 # F: 第0年豆类种植量 (单位: 亩)
26
27 # 读取Excel文件
28 df = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/Q1.xlsx')
29
30 # 剔除不需要的作物
31 crops_to_remove = ['水稻', '大白菜', '红萝卜', '白萝卜', '榆黄菇', '香菇', '白灵菇',
32                    , '羊肚菌']
33 crop_info = df[~df['作物名称'].isin(crops_to_remove)]
34
35 # 使用Q1中的需求量和价格数据
36 crop_demand = crop_info['需求量'].values
37 crop_prices = crop_info['非智慧单价'].values
38 crop_price_f = crop_info['智慧单价'].values
39
40 # 作物编号设定: 所有作物编号减去1, 使编号从0开始
41 grain_crops = list(range(0, 15)) # 粮食类作物编号为0-14
42 bean_crops = list(range(0, 5)) # 豆类作物编号为0-4
```

```

42 vegetable_crops = list(range(15, 33)) # 蔬菜类作物编号为15-32
43 bean_crops_DEF = list(range(15, 18)) # DEF地块豆类作物编号为15-17
44
45 # 获取不同土地类型的作物产量和成本数据
46 crop_yield_by_land = {
47     'A': crop_info['平旱地产量'].values[:15],
48     'B': crop_info['梯田产量'].values[:15],
49     'C': crop_info['山坡地产量'].values[:15],
50 }
51
52 crop_cost_by_land = {
53     'A': crop_info['平旱地成本'].values[:15],
54     'B': crop_info['梯田成本'].values[:15],
55     'C': crop_info['山坡地成本'].values[:15],
56 }
57
58 crop_yield_by_land_DEF = {
59     'D': crop_info['水浇地产量'].values[15:33],
60     'E': crop_info['普通大棚产量'].values[15:33],
61     'F': crop_info['普通大棚产量'].values[15:33],
62 }
63
64 crop_cost_by_land_DEF = {
65     'D': crop_info['水浇地成本'].values[15:33],
66     'E': crop_info['普通大棚成本'].values[15:33],
67     'F': crop_info['普通大棚成本'].values[15:33],
68 }
69
70 # 模拟退火算法 1: 处理ABC (粮食类作物)
71 def simulated_annealing_ABC(init_temp, min_temp, cooling_rate, max_iter):
72     """模拟退火求解农业规划问题 (ABC粮食作物)"""
73     current_solution = np.zeros((num_lands_ABC, 15, num_years)) # 粮食类作物 0-14
74
75     for land_index, land_name in enumerate(['A', 'B', 'C']):
76         for year in range(num_years):
77             crop = random.choice(grain_crops)
78             max_area = areas[land_name] # 受土地面积限制
79             current_solution[land_index, crop, year] = round(random.uniform(0,
80 max_area), 1) # 保留一位小数
81
82             # 检查并限制每年的种植面积不能超过土地总面积
83             total_area_for_year = np.sum(current_solution[land_index, :, year])
84             if total_area_for_year > areas[land_name]:
85                 # 调整种植面积比例, 确保不超过土地面积
86                 current_solution[land_index, :, year] *= areas[land_name] /
total_area_for_year

```

```

87     # 保持豆类作物三年种植约束
88     for year in range(2, num_years):
89         bean_area_sum = np.sum(current_solution[land_index, bean_crops, year])
+ \
90             np.sum(current_solution[land_index, bean_crops, year -
1])) + \
91             np.sum(current_solution[land_index, bean_crops, year -
2]))
92         if land_name == 'A':
93             bean_area_sum += n1_A
94         elif land_name == 'B':
95             bean_area_sum += n1_B
96         elif land_name == 'C':
97             bean_area_sum += n1_C
98
99         if bean_area_sum < areas[land_name]:
100             bean_area_deficit = areas[land_name] - bean_area_sum
101             selected_bean_crop = random.choice(bean_crops)
102             current_solution[land_index, selected_bean_crop, year] += round(
bean_area_deficit, 1)
103
104     best_solution = np.copy(current_solution)
105     current_profit = calculate_profit_ABC(current_solution)
106     best_profit = current_profit
107
108     temp = init_temp
109     profits = []
110
111     while temp > min_temp:
112         for _ in range(max_iter):
113             new_solution = np.copy(current_solution)
114
115             for land_index, land_name in enumerate(['A', 'B', 'C']):
116                 for year in range(num_years):
117                     crop = random.choice(grain_crops)
118                     max_area = areas[land_name]
119                     new_solution[land_index, crop, year] = round(random.uniform(0,
max_area), 1)
120
121                 # 检查并限制每年的种植面积不能超过土地总面积
122                 total_area_for_year = np.sum(new_solution[land_index, :, year])
123                 if total_area_for_year > areas[land_name]:
124                     new_solution[land_index, :, year] *= areas[land_name] /
total_area_for_year
125
126                 # 加入重复种植限制：连续两年同种作物的种植面积之和不能超过土地
总面积

```

```

127         if year > 0:
128             if new_solution[land_index, crop, year] + new_solution[
land_index, crop, year - 1] > areas[land_name]:
129                 new_solution[land_index, crop, year] = areas[land_name]
- new_solution[land_index, crop, year - 1]
130
131         # 豆类作物三年内种植约束
132         if year > 1:
133             bean_area_sum = np.sum(new_solution[land_index, bean_crops,
year]) + \
134                                     np.sum(new_solution[land_index, bean_crops,
year - 1]) + \
135                                     np.sum(new_solution[land_index, bean_crops,
year - 2])
136
137             if land_name == 'A':
138                 bean_area_sum += n1_A
139             elif land_name == 'B':
140                 bean_area_sum += n1_B
141             elif land_name == 'C':
142                 bean_area_sum += n1_C
143
144             if bean_area_sum < areas[land_name]:
145                 bean_area_deficit = areas[land_name] - bean_area_sum
146                 selected_bean_crop = random.choice(bean_crops)
147                 new_solution[land_index, selected_bean_crop, year] +=
round(bean_area_deficit, 1)
148
149         for year in range(num_years):
150             total_area_for_year = np.sum(new_solution[land_index, :, year])
151             if total_area_for_year > areas[land_name]:
152                 new_solution[land_index, :, year] *= (areas[land_name] /
total_area_for_year)
153
154         new_profit = calculate_profit_ABC(new_solution)
155
156         if new_profit > current_profit:
157             current_solution = new_solution
158             current_profit = new_profit
159             if new_profit > best_profit:
160                 best_solution = np.copy(new_solution)
161                 best_profit = new_profit
162         else:
163             delta = new_profit - current_profit
164             if np.exp(delta / temp) > random.uniform(0, 1):
165                 current_solution = new_solution
166                 current_profit = new_profit

```

```

167
168     profits.append(best_profit)
169     temp *= cooling_rate
170
171     return best_solution, best_profit, profits
172
173 # 模拟退火算法 2: 处理DEF (蔬菜类作物)
174 def simulated_annealing_DEF(init_temp, min_temp, cooling_rate, max_iter):
175     """模拟退火求解农业规划问题 (DEF蔬菜作物) """
176     current_solution = np.zeros((num_lands_DEF, 18, num_years)) # 蔬菜类作物 15-32
177                                     (共18种)
178
179     for land_index, land_name in enumerate(['D', 'E', 'F']):
180         for year in range(num_years):
181             crop = random.choice(vegetable_crops) - 15 # 索引从0开始
182             max_area = min(crop_demand[crop + 15] / crop_yield_by_land_DEF[
183                 land_name][crop], areas_DEF[land_name])
184
185             # 在E和F地块确保种植面积大于0.3, 且保留一位小数
186             if land_name in ['E', 'F']:
187                 area = round(random.uniform(0.3, max_area), 1)
188                 current_solution[land_index, crop, year] = area
189             else:
190                 current_solution[land_index, crop, year] = round(random.uniform(0,
191                 max_area), 1)
192
193             # 确保每年的种植面积不超过土地面积, 且D地块在豆类判断时面积从67亩变为
194             109亩
195             if land_name == 'D' and crop in bean_crops_DEF:
196                 total_area_for_year = np.sum(current_solution[land_index, :, year])
197                 if total_area_for_year > 109: # D地块的面积调整为109亩
198                     current_solution[land_index, :, year] *= 109 /
199                     total_area_for_year
200             else:
201                 total_area_for_year = np.sum(current_solution[land_index, :, year])
202                 if total_area_for_year > areas_DEF[land_name]:
203                     current_solution[land_index, :, year] *= areas_DEF[land_name] /
204                     total_area_for_year
205
206     best_solution = np.copy(current_solution)
207     current_profit = calculate_profit_DEF(current_solution)
208     best_profit = current_profit
209
210     temp = init_temp
211     profits = []
212
213     while temp > min_temp:

```

```

208     for _ in range(max_iter):
209         new_solution = np.copy(current_solution)
210
211         for land_index, land_name in enumerate(['D', 'E', 'F']):
212             for year in range(num_years):
213                 crop = random.choice(vegetable_crops) - 15
214                 remaining_demand = max(0, crop_demand[crop + 15] - np.sum(
new_solution[land_index, crop, :year]))
215                 max_area = min(remaining_demand / crop_yield_by_land_DEF[
land_name][crop], areas_DEF[land_name])
216
217                 # 在E和F地块确保种植面积大于0.3，且保留一位小数
218                 if land_name in ['E', 'F']:
219                     area = round(random.uniform(0.3, max_area), 1)
220                     new_solution[land_index, crop, year] = area
221                 else:
222                     new_solution[land_index, crop, year] = round(random.uniform
(0, max_area), 1)
223
224                 # 确保每年的种植面积不超过土地面积，且D地块在豆类判断时面积调整
为109亩
225                 if land_name == 'D' and crop in bean_crops_DEF:
226                     total_area_for_year = np.sum(new_solution[land_index, :,
year])
227                     if total_area_for_year > 109: # D地块面积调整为109亩
228                         new_solution[land_index, :, year] *= 109 /
total_area_for_year
229                     else:
230                         total_area_for_year = np.sum(new_solution[land_index, :,
year])
231                     if total_area_for_year > areas_DEF[land_name]:
232                         new_solution[land_index, :, year] *= areas_DEF[
land_name] / total_area_for_year
233
234                 new_profit = calculate_profit_DEF(new_solution)
235
236                 if new_profit > current_profit:
237                     current_solution = new_solution
238                     current_profit = new_profit
239                 if new_profit > best_profit:
240                     best_solution = np.copy(new_solution)
241                     best_profit = new_profit
242             else:
243                 delta = new_profit - current_profit
244                 if np.exp(delta / temp) > random.uniform(0, 1):
245                     current_solution = new_solution
246                     current_profit = new_profit

```



```

247
248     profits.append(best_profit)
249     temp *= cooling_rate
250
251     return best_solution, best_profit, profits
252
253
254 # 定义ABC的利润计算函数
255 def calculate_profit_ABC(solution):
256     """计算ABC粮食类作物的总利润"""
257     total_profit = 0
258     for land_index, land_name in enumerate(['A', 'B', 'C']):
259         for crop in range(15):
260             for year in range(num_years):
261                 total_yield = solution[land_index, crop, year] * crop_yield_by_land[
262                     land_name][crop]
263                 total_cost = solution[land_index, crop, year] * crop_cost_by_land[
264                     land_name][crop]
265
266                 # 保留一位小数进行收益和成本计算
267                 total_yield = round(total_yield, 1)
268                 total_cost = round(total_cost, 1)
269
270                 # 计算收益，滞销部分为0
271                 if total_yield > crop_demand[crop]:
272                     revenue = crop_demand[crop] * crop_prices[crop] # 按需求量计算
273                     wastage = (total_yield - crop_demand[crop]) * 0 # 滞销部分，收
274                     益为0
275                 else:
276                     revenue = total_yield * crop_prices[crop] # 正常收益
277
278                 # 总利润 = 收益 - 成本
279                 total_profit += round(revenue - total_cost, 1)
280
281     return total_profit
282
283
284 # 定义DEF的利润计算函数（F地块使用不同的销售价格）
285 def calculate_profit_DEF(solution):
286     """计算DEF蔬菜类作物的总利润"""
287     total_profit = 0
288     crop_price_f = crop_info['智慧单价'].values
289
290     for land_index, land_name in enumerate(['D', 'E', 'F']):
291         for crop in range(18):
292             for year in range(num_years):

```

```

290         total_yield = solution[land_index, crop, year] *
crop_yield_by_land_DEF[land_name][crop]
291         total_cost = solution[land_index, crop, year] *
crop_cost_by_land_DEF[land_name][crop]
292
293         if land_name == 'F':
294             price = crop_price_f[crop]
295         else:
296             price = crop_prices[crop + 15]
297
298         if total_yield > crop_demand[crop + 15]:
299             revenue = crop_demand[crop + 15] * price
300             wastage = (total_yield - crop_demand[crop + 15]) * 0
301         else:
302             revenue = total_yield * price
303
304         total_profit += revenue - total_cost
305     return total_profit
306
307 # 储存
308 def save_solution_to_excel(best_solution_ABC, best_solution_DEF, crop_info,
num_years):
309     writer = pd.ExcelWriter('planting_solutions_by_land.xlsx', engine='openpyxl')
310     crops_names = crop_info['作物名称'].values
311
312     # 创建ABC地块的表格
313     for i, land_name in enumerate(['A', 'B', 'C']):
314         # 向下取整并保留一位小数
315         abc_data = np.floor(best_solution_ABC[i, :, :] * 10) / 10
316         df = pd.DataFrame(abc_data.T, columns=crops_names[:15], index=[f'{2024+year
}年' for year in range(num_years)])
317         df.to_excel(writer, sheet_name=f'Land_{land_name}', index_label='Year')
318
319     # 创建DEF地块的表格
320     for i, land_name in enumerate(['D', 'E', 'F']):
321         # 向下取整并保留一位小数
322         def_data = np.floor(best_solution_DEF[i, :, :] * 10) / 10
323         df = pd.DataFrame(def_data.T, columns=crops_names[15:33], index=[f'{2024+
year}年' for year in range(num_years)])
324         df.to_excel(writer, sheet_name=f'Land_{land_name}', index_label='Year')
325
326     writer.close()
327
328
329 # 参数设置
330 init_temp = 1000
331 min_temp = 1

```

```

332 cooling_rate = 0.95
333 max_iter = 1000
334
335 # 运行ABC地块（粮食作物）的模拟退火
336 best_solution_ABC, best_profit_ABC, profits_ABC = simulated_annealing_ABC(init_temp
    , min_temp, cooling_rate, max_iter)
337
338 # 输出最佳方案的总利润
339 print(f"ABC地块最佳方案的总利润: {best_profit_ABC}")
340
341 # 运行DEF地块（蔬菜作物）的模拟退火
342 best_solution_DEF, best_profit_DEF, profits_DEF = simulated_annealing_DEF(init_temp
    , min_temp, cooling_rate, max_iter)
343
344 # 输出最佳方案的总利润
345 print(f"DEF地块最佳方案的总利润: {best_profit_DEF}")
346
347 # 保存最佳种植方案到Excel文件
348 save_solution_to_excel(best_solution_ABC, best_solution_DEF, crop_info, num_years)
349
350 # 总利润
351 best_profit_total = best_profit_ABC + best_profit_DEF
352 print(f"Total best profit: {best_profit_total}")
353
354 # 如果 profits_ABC 的长度小于 profits_DEF 的长度，使用 profits_ABC 的最后一个值补齐
355 if len(profits_ABC) < len(profits_DEF):
356     difference = len(profits_DEF) - len(profits_ABC)
357     profits_ABC.extend([profits_ABC[-1]] * difference)
358
359 # 如果 profits_DEF 的长度小于 profits_ABC 的长度，使用 profits_DEF 的最后一个值补齐
360 if len(profits_DEF) < len(profits_ABC):
361     difference = len(profits_ABC) - len(profits_DEF)
362     profits_DEF.extend([profits_DEF[-1]] * difference)
363
364 # 将 profits_ABC 和 profits_DEF 进行合并，计算总的 profits
365 profits = [x + y for x, y in zip(profits_ABC, profits_DEF)]
366
367 # 调整图片尺寸
368 plt.figure(figsize=(5, 4)) # 将图片尺寸设置为宽 6 英寸，高 4 英寸
369
370 # 绘制三条线，使用图片中的颜色
371 plt.plot(profits_ABC, label='ABC Profits', color=(33/255, 49/255, 140/255),
    linewidth=2) # 第一种颜色
372 plt.plot(profits_DEF, label='DEF Profits', color=(48/255, 165/255, 194/255),
    linewidth=2) # 第二种颜色
373 plt.plot(profits, label='Total Profits', color=(149/255, 213/255, 184/255),
    linewidth=2.5) # 第三种颜色

```

```

374
375 # 设置x轴从0开始
376 plt.xlim(left=0)
377
378 # 添加网格
379 plt.grid(True)
380
381 # 将图例放在图片下方
382 plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.1), fancybox=True, shadow=
    True, ncol=3)
383
384 # 添加标题和坐标轴标签
385 plt.xlabel("Iterations")
386 plt.ylabel("Profit")
387
388 # 调整图表布局，使图例不遮挡图表
389 plt.tight_layout()
390
391 # 显示图表
392 plt.show()
393
394 # 计算总的最佳利润
395 best_profit_total = best_profit_ABC + best_profit_DEF
396 print(f"Total best profit: {best_profit_total}")

```

2. qn_2_ 最优地块分配.py

```

1  -*- coding: utf-8 -*-
2
3  import pandas as pd
4  import numpy as np
5  import random
6  import matplotlib.pyplot as plt
7
8  # 读取地块信息和种植方案
9  df_A = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_A')
10 df_B = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_B')
11 df_C = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_C')
12 df_D = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_D')
13 df_E = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_E')
14 df_F = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/
    planting_solutions_by_land.xlsx', sheet_name='Land_F')
15

```

```

16 area = pd.read_excel('C:/Users/86188/Desktop/2024数模/问题一/Q1step3.xlsx')
17 area_A = area[area['地块类型'] == '平旱地']
18 area_B = area[area['地块类型'] == '梯田']
19 area_C = area[area['地块类型'] == '山坡地']
20 area_D = area[area['地块类型'] == '水浇地']
21 area_E = area[area['地块类型'] == '普通大棚']
22 area_F = area[area['地块类型'] == '智慧大棚']
23
24 # 获取相关参数
25 years = df_A['Year'].values
26 crops = df_A.columns[1:]
27 num_years = len(years)
28 crop_list = list(crops)
29 planting_data = df_A.iloc[:, 1:].values # 提取各年份的种植数据
30 bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆']
31 other_crops = [crop for crop in crop_list if crop not in bean_crops]
32
33 # A地块参数
34 areas_A = area_A['地块面积/亩'].tolist()
35 initial_bean_areas = area_A['2023年豆类种植面积'].tolist()
36 num_lands = len(areas_A)
37
38 # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
39 def initialize_solution():
40     solution = np.zeros((num_lands, len(crop_list), num_years))
41
42     # 先分配豆类作物
43     for land_index in range(num_lands):
44         for year in range(num_years):
45             # 保证豆类作物在三年内至少种植一次，且避免重茬种植
46             if year >= 2:
47                 total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
48                     land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
49                 if total_bean_area < areas_A[land_index]:
50                     # 随机选择豆类作物
51                     selected_bean = random.choice(bean_crops)
52                     available_area = areas_A[land_index] - np.sum(solution[
53                         land_index, :, year])
54                     if available_area > 0:
55                         solution[land_index, crop_list.index(selected_bean), year]
56                         = min(available_area, areas_A[land_index])
57
58     return solution
59
60 # 确保豆类作物满足三年种植约束（每次分配时都检验）
61 def enforce_bean_constraint(solution):
62     for land_index in range(num_lands):

```

```

60     for year in range(2, num_years): # 从第2年开始检查三年内的种植面积
61         # 第0年豆类种植面积已知，计算三年内的豆类种植面积
62         total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
63         if total_bean_area < areas_A[land_index]:
64             # 需要调整豆类作物的种植面积，以满足面积要求
65             deficit = areas_A[land_index] - total_bean_area
66             # 随机选择一个豆类作物并增加其种植面积
67             selected_bean = random.choice(bean_crops)
68             selected_year = random.randint(year-2, year) # 随机选择三年中的某
一年
69             available_area = areas_A[land_index] - np.sum(solution[land_index,
:, selected_year]) # 剩余可用面积
70             if available_area > 0:
71                 solution[land_index, crop_list.index(selected_bean),
selected_year] += min(deficit, available_area)
72     return solution
73
74 # 避免连续重茬种植的约束（相邻两年种植同种作物的总面积不得超过该地块面积）
75 def enforce_rotation_constraint(solution):
76     for land_index in range(num_lands):
77         for year in range(1, num_years):
78             for crop_index in range(len(crop_list)):
79                 crop_area_year1 = solution[land_index, crop_index, year]
80                 crop_area_year0 = solution[land_index, crop_index, year - 1]
81                 if crop_area_year1 + crop_area_year0 > areas_A[land_index]:
82                     # 调整当年的种植面积，确保不超过相邻两年地块面积之和
83                     adjustment = crop_area_year1 + crop_area_year0 - areas_A[
land_index]
84                     if crop_area_year1 > adjustment:
85                         solution[land_index, crop_index, year] -= adjustment
86                     else:
87                         solution[land_index, crop_index, year - 1] -= (adjustment -
crop_area_year1)
88     return solution
89
90 # 分配其他作物，避免重茬，尽量一年内种植一个作物，同时验证地块面积约束
91 def allocate_other_crops(solution):
92     for land_index in range(num_lands):
93         for year in range(num_years):
94             remaining_area = areas_A[land_index] - np.sum(solution[land_index, :,
year]) # 剩余可种植面积
95             if remaining_area > 0:
96                 selected_crop = random.choice(other_crops) # 随机选择其他作物
97                 solution[land_index, crop_list.index(selected_crop), year] = min(
remaining_area, areas_A[land_index]) # 确保分配不超过地块面积
98     return solution

```

```

99
100 # 确保植物的种植面积不为负
101 def enforce_non_negative_area(solution):
102     # 遍历所有地块、所有年份和所有作物
103     for land_index in range(num_lands):
104         for year in range(num_years):
105             for crop_index in range(len(crop_list)):
106                 if solution[land_index, crop_index, year] < 0:
107                     solution[land_index, crop_index, year] = 0
108     return solution
109
110 # 计算目标函数：种植的作物种类数量之和
111 def calculate_objective(solution):
112     total_crops = 0
113     for land_index in range(num_lands):
114         for year in range(num_years):
115             # 统计每块地每年的种植种类数量
116             planted_crops = np.sum(solution[land_index, :, year] > 0)
117             total_crops += planted_crops
118     return total_crops
119
120 # 模拟退火算法，保存最新的解
121 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
122     max_iter):
123     current_solution = initialize_solution()
124     # 确保满足豆类种植约束
125     current_solution = enforce_bean_constraint(current_solution)
126     current_solution = enforce_rotation_constraint(current_solution) # 新增重茬种植限制
127     current_solution = allocate_other_crops(current_solution) # 分配其他作物
128     current_objective = calculate_objective(current_solution)
129     best_solution = np.copy(current_solution)
130     latest_solution = np.copy(current_solution) # 保存最新解
131     best_objective = current_objective
132
133     temp = init_temp
134     objectives = [current_objective] # 存储每次迭代的目标函数值
135
136     iteration_count = 0 # 记录迭代次数
137
138     while temp > min_temp:
139         for _ in range(max_iter):
140             iteration_count += 1
141             new_solution = np.copy(current_solution)
142
143             # 随机调整一个地块某年的作物种植面积
144             land_index = random.randint(0, num_lands - 1)

```

```

144     year = random.randint(0, num_years - 1)
145     crop = random.choice(crop_list)
146     adjustment = round(random.uniform(-5, 5), 1)
147     new_solution[land_index, crop_list.index(crop), year] += adjustment
148
149     # 确保每个地块的总种植面积不超过地块面积
150     total_area_for_year = np.sum(new_solution[land_index, :, year])
151     if total_area_for_year > areas_A[land_index]:
152         # 调整作物面积以确保总面积不超过地块面积
153         new_solution[land_index, :, year] *= areas_A[land_index] /
total_area_for_year
154
155     # 确保满足豆类种植约束
156     new_solution = enforce_bean_constraint(new_solution)
157     # 确保避免连续重茬种植
158     new_solution = enforce_rotation_constraint(new_solution)
159
160     # 确保种植面积不为负
161     new_solution = enforce_non_negative_area(new_solution)
162
163     # 重新计算目标函数
164     new_objective = calculate_objective(new_solution)
165
166     # 接受新解的策略
167     if new_objective < current_objective:
168         current_solution = new_solution
169         current_objective = new_objective
170         if new_objective < best_objective:
171             best_solution = np.copy(new_solution)
172             best_objective = new_objective
173     else:
174         delta = new_objective - current_objective
175         if np.exp(-delta / temp) > random.uniform(0, 1):
176             current_solution = new_solution
177             current_objective = new_objective
178
179     # 保存最新的解
180     latest_solution = np.copy(current_solution)
181
182     objectives.append(current_objective) # 保存每次迭代后的目标函数值
183
184     temp *= cooling_rate
185
186     print(f"总迭代次数: {iteration_count}")
187     return best_solution, latest_solution, best_objective, objectives
188
189 # 可视化迭代过程中目标函数值的变化

```



```

190 def visualize_objective_iterations(objectives):
191     plt.figure(figsize=(5, 5))
192     plt.plot(objectives, label='Objective Value Over Iterations')
193     plt.xlabel('Iteration')
194     plt.ylabel('Objective Value')
195     plt.legend()
196     plt.grid(True)
197     plt.xticks([]) # 移除 x 轴上的刻度标签
198     plt.show()
199
200 # 保存最新的解到Excel文件
201 def save_solution_to_excel(solution, filename):
202     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
203         for year in range(num_years):
204             # 为每年创建一个sheet
205             sheet_name = f'Year_{years[year]}'
206             df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
207 index=[f'A{land+1}' for land in range(num_lands)])
208             df_solution.to_excel(writer, sheet_name=sheet_name)
209             print(f"最新解已保存到 {filename}")
210
211 # 参数设置
212 init_temp = 1000
213 min_temp = 1
214 cooling_rate = 0.98 # 放慢冷却速度
215 max_iter = 5000 # 增加迭代次数
216
217 # 运行模拟退火并获取每次迭代的目标函数值
218 best_solution, latest_solution, best_objective, objectives =
219     simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
220 max_iter)
221
222 # 可视化
223 visualize_objective_iterations(objectives)
224 # print(f"A最佳方案的作物种类数量之和: {latest_solution}")
225
226 # 调用保存函数，将最新解保存到文件
227 save_solution_to_excel(latest_solution, 'latest_solution_A.xlsx')
228
229 # 获取相关参数
230 years = df_B['Year'].values
231 crops = df_B.columns[1:]
232 num_years = len(years)
233 crop_list = list(crops)
234
235 planting_data = df_B.iloc[:, 1:].values # 提取各年份的种植数据
236 bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆']
237 other_crops = [crop for crop in crop_list if crop not in bean_crops]

```

```

234
235 # B地块参数
236 areas_B = area_B['地块面积/亩'].tolist()
237 initial_bean_areas = area_B['2023年豆类种植面积'].tolist()
238 num_lands = len(areas_B)
239
240 # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
241 def initialize_solution():
242     solution = np.zeros((num_lands, len(crop_list), num_years))
243
244     # 先分配豆类作物
245     for land_index in range(num_lands):
246         for year in range(num_years):
247             # 保证豆类作物在三年内至少种植一次，且避免重茬种植
248             if year >= 2:
249                 total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
250 land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
251                 if total_bean_area < areas_B[land_index]:
252                     # 随机选择豆类作物
253                     selected_bean = random.choice(bean_crops)
254                     available_area = areas_B[land_index] - np.sum(solution[
255 land_index, :, year])
256                     if available_area > 0:
257                         solution[land_index, crop_list.index(selected_bean), year]
258 = min(available_area, areas_B[land_index])
259
260     return solution
261
262 # 确保豆类作物满足三年种植约束（每次分配时都检验）
263 def enforce_bean_constraint(solution):
264     for land_index in range(num_lands):
265         for year in range(2, num_years): # 从第2年开始检查三年内的种植面积
266             # 第0年豆类种植面积已知，计算三年内的豆类种植面积
267             total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
268 land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
269             if total_bean_area < areas_B[land_index]:
270                 # 需要调整豆类作物的种植面积，以满足面积要求
271                 deficit = areas_B[land_index] - total_bean_area
272                 # 随机选择一个豆类作物并增加其种植面积
273                 selected_bean = random.choice(bean_crops)
274                 selected_year = random.randint(year-2, year) # 随机选择三年中的某
275 一年
276                 available_area = areas_B[land_index] - np.sum(solution[land_index,
277 :, selected_year]) # 剩余可用面积
278                 if available_area > 0:
279                     solution[land_index, crop_list.index(selected_bean),
280 selected_year] += min(deficit, available_area)

```

```

274     return solution
275
276 # 避免连续重茬种植的约束（相邻两年种植同种作物的总面积不得超过该地块面积）
277 def enforce_rotation_constraint(solution):
278     for land_index in range(num_lands):
279         for year in range(1, num_years):
280             for crop_index in range(len(crop_list)):
281                 crop_area_year1 = solution[land_index, crop_index, year]
282                 crop_area_year0 = solution[land_index, crop_index, year - 1]
283                 if crop_area_year1 + crop_area_year0 > areas_B[land_index]:
284                     # 调整当年的种植面积，确保不超过相邻两年地块面积之和
285                     adjustment = crop_area_year1 + crop_area_year0 - areas_B[
land_index]
286                     if crop_area_year1 > adjustment:
287                         solution[land_index, crop_index, year] -= adjustment
288                     else:
289                         solution[land_index, crop_index, year - 1] -= (adjustment -
crop_area_year1)
290     return solution
291
292 # 分配其他作物，避免重茬，尽量一年内种植一个作物，同时验证地块面积约束
293 def allocate_other_crops(solution):
294     for land_index in range(num_lands):
295         for year in range(num_years):
296             remaining_area = areas_B[land_index] - np.sum(solution[land_index, :,
year]) # 剩余可种植面积
297             if remaining_area > 0:
298                 selected_crop = random.choice(other_crops) # 随机选择其他作物
299                 solution[land_index, crop_list.index(selected_crop), year] = min(
remaining_area, areas_B[land_index]) # 确保分配不超过地块面积
300     return solution
301
302 # 确保植物的种植面积不为负
303 def enforce_non_negative_area(solution):
304     # 遍历所有地块、所有年份和所有作物
305     for land_index in range(num_lands):
306         for year in range(num_years):
307             for crop_index in range(len(crop_list)):
308                 if solution[land_index, crop_index, year] < 0:
309                     solution[land_index, crop_index, year] = 0
310     return solution
311
312 # 计算目标函数：种植的作物种类数量之和
313 def calculate_objective(solution):
314     total_crops = 0
315     for land_index in range(num_lands):
316         for year in range(num_years):

```

```

317         # 统计每块地每年的种植种类数量
318         planted_crops = np.sum(solution[land_index, :, year] > 0)
319         total_crops += planted_crops
320     return total_crops
321
322 # 模拟退火算法，保存最新的解
323 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
324     max_iter):
325     current_solution = initialize_solution()
326     # 确保满足豆类种植约束
327     current_solution = enforce_bean_constraint(current_solution)
328     current_solution = enforce_rotation_constraint(current_solution) # 新增重茬种植限制
329     current_solution = allocate_other_crops(current_solution) # 分配其他作物
330     current_objective = calculate_objective(current_solution)
331     best_solution = np.copy(current_solution)
332     latest_solution = np.copy(current_solution) # 保存最新解
333     best_objective = current_objective
334
335     temp = init_temp
336     objectives = [current_objective] # 存储每次迭代的目标函数值
337
338     iteration_count = 0 # 记录迭代次数
339
340     while temp > min_temp:
341         for _ in range(max_iter):
342             iteration_count += 1
343             new_solution = np.copy(current_solution)
344
345             # 随机调整一个地块某年的作物种植面积
346             land_index = random.randint(0, num_lands - 1)
347             year = random.randint(0, num_years - 1)
348             crop = random.choice(crop_list)
349             adjustment = round(random.uniform(-5, 5), 1)
350             new_solution[land_index, crop_list.index(crop), year] += adjustment
351
352             # 确保每个地块的总种植面积不超过地块面积
353             total_area_for_year = np.sum(new_solution[land_index, :, year])
354             if total_area_for_year > areas_B[land_index]:
355                 # 调整作物面积以确保总面积不超过地块面积
356                 new_solution[land_index, :, year] *= areas_B[land_index] /
357                 total_area_for_year
358
359             # 确保满足豆类种植约束
360             new_solution = enforce_bean_constraint(new_solution)
361             # 确保避免连续重茬种植
362             new_solution = enforce_rotation_constraint(new_solution)

```

```

361
362     # 确保种植面积不为负
363     new_solution = enforce_non_negative_area(new_solution)
364
365     # 重新计算目标函数
366     new_objective = calculate_objective(new_solution)
367
368     # 接受新解的策略
369     if new_objective < current_objective:
370         current_solution = new_solution
371         current_objective = new_objective
372         if new_objective < best_objective:
373             best_solution = np.copy(new_solution)
374             best_objective = new_objective
375     else:
376         delta = new_objective - current_objective
377         if np.exp(-delta / temp) > random.uniform(0, 1):
378             current_solution = new_solution
379             current_objective = new_objective
380
381     # 保存最新的解
382     latest_solution = np.copy(current_solution)
383
384     objectives.append(current_objective) # 保存每次迭代后的目标函数值
385
386     temp *= cooling_rate
387
388     print(f"总迭代次数: {iteration_count}")
389     return best_solution, latest_solution, best_objective, objectives
390
391 # 可视化迭代过程中目标函数值的变化
392 def visualize_objective_iterations(objectives):
393     plt.figure(figsize=(5, 5))
394     plt.plot(objectives, label='Objective Value Over Iterations')
395     plt.xlabel('Iteration')
396     plt.ylabel('Objective Value')
397     plt.legend()
398     plt.grid(True)
399     plt.xticks([]) # 移除 x 轴上的刻度标签
400     plt.show()
401
402 # 保存最新的解到Excel文件
403 def save_solution_to_excel(solution, filename):
404     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
405         for year in range(num_years):
406             # 为每年创建一个sheet
407             sheet_name = f'Year_{years[year]}'

```

```

408         df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
index=[f'B{land+1}' for land in range(num_lands)])
409         df_solution.to_excel(writer, sheet_name=sheet_name)
410         print(f"最新解已保存到 {filename}")
411
412     # 参数设置
413     init_temp = 1000
414     min_temp = 1
415     cooling_rate = 0.98 # 放慢冷却速度
416     max_iter = 5000 # 增加迭代次数
417
418     # 运行模拟退火并获取每次迭代的目标函数值
419     best_solution, latest_solution, best_objective, objectives =
        simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
        max_iter)
420
421     # 可视化
422     visualize_objective_iterations(objectives)
423     # print(f"B最佳方案的作物种类数量之和: {latest_solution}")
424
425     # 调用保存函数，将最新解保存到文件
426     save_solution_to_excel(latest_solution, 'latest_solution_B.xlsx')
427
428     # 获取相关参数
429     years = df_C['Year'].values
430     crops = df_C.columns[1:]
431     num_years = len(years)
432     crop_list = list(crops)
433     planting_data = df_C.iloc[:, 1:].values # 提取各年份的种植数据
434     bean_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆']
435     other_crops = [crop for crop in crop_list if crop not in bean_crops]
436
437     # C地块参数
438     areas_C = area_C['地块面积/亩'].tolist()
439     initial_bean_areas = area_C['2023年豆类种植面积'].tolist()
440     num_lands = len(areas_C)
441
442     # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
443     def initialize_solution():
444         solution = np.zeros((num_lands, len(crop_list), num_years))
445
446         # 先分配豆类作物
447         for land_index in range(num_lands):
448             for year in range(num_years):
449                 # 保证豆类作物在三年内至少种植一次，且避免重茬种植
450                 if year >= 2:
451                     total_bean_area = initial_bean_areas[land_index] + np.sum(solution[

```

```

land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
452         if total_bean_area < areas_C[land_index]:
453             # 随机选择豆类作物
454             selected_bean = random.choice(bean_crops)
455             available_area = areas_C[land_index] - np.sum(solution[
land_index, :, year])
456             if available_area > 0:
457                 solution[land_index, crop_list.index(selected_bean), year]
= min(available_area, areas_C[land_index])
458
459         return solution
460
461 # 确保豆类作物满足三年种植约束（每次分配时都检验）
462 def enforce_bean_constraint(solution):
463     for land_index in range(num_lands):
464         for year in range(2, num_years): # 从第2年开始检查三年内的种植面积
465             # 第0年豆类种植面积已知，计算三年内的豆类种植面积
466             total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
467             if total_bean_area < areas_C[land_index]:
468                 # 需要调整豆类作物的种植面积，以满足面积要求
469                 deficit = areas_C[land_index] - total_bean_area
470                 # 随机选择一个豆类作物并增加其种植面积
471                 selected_bean = random.choice(bean_crops)
472                 selected_year = random.randint(year-2, year) # 随机选择三年中的某
一年
473                 available_area = areas_C[land_index] - np.sum(solution[land_index,
:, selected_year]) # 剩余可用面积
474                 if available_area > 0:
475                     solution[land_index, crop_list.index(selected_bean),
selected_year] += min(deficit, available_area)
476             return solution
477
478 # 避免连续重茬种植的约束（相邻两年种植同种作物的总面积不得超过该地块面积）
479 def enforce_rotation_constraint(solution):
480     for land_index in range(num_lands):
481         for year in range(1, num_years):
482             for crop_index in range(len(crop_list)):
483                 crop_area_year1 = solution[land_index, crop_index, year]
484                 crop_area_year0 = solution[land_index, crop_index, year - 1]
485                 if crop_area_year1 + crop_area_year0 > areas_C[land_index]:
486                     # 调整当年的种植面积，确保不超过相邻两年地块面积之和
487                     adjustment = crop_area_year1 + crop_area_year0 - areas_C[
land_index]
488                     if crop_area_year1 > adjustment:
489                         solution[land_index, crop_index, year] -= adjustment
490                     else:

```

```

491         solution[land_index, crop_index, year - 1] -= (adjustment -
crop_area_year1)
492     return solution
493
494 # 分配其他作物，避免重茬，尽量一年内种植一个作物，同时验证地块面积约束
495 def allocate_other_crops(solution):
496     for land_index in range(num_lands):
497         for year in range(num_years):
498             remaining_area = areas_C[land_index] - np.sum(solution[land_index, :,
year]) # 剩余可种植面积
499             if remaining_area > 0:
500                 selected_crop = random.choice(other_crops) # 随机选择其他作物
501                 solution[land_index, crop_list.index(selected_crop), year] = min(
remaining_area, areas_C[land_index]) # 确保分配不超过地块面积
502     return solution
503
504 # 确保植物的种植面积不为负
505 def enforce_non_negative_area(solution):
506     # 遍历所有地块、所有年份和所有作物
507     for land_index in range(num_lands):
508         for year in range(num_years):
509             for crop_index in range(len(crop_list)):
510                 if solution[land_index, crop_index, year] < 0:
511                     solution[land_index, crop_index, year] = 0
512     return solution
513
514 # 计算目标函数：种植的作物种类数量之和
515 def calculate_objective(solution):
516     total_crops = 0
517     for land_index in range(num_lands):
518         for year in range(num_years):
519             # 统计每块地每年的种植种类数量
520             planted_crops = np.sum(solution[land_index, :, year] > 0)
521             total_crops += planted_crops
522     return total_crops
523
524 # 模拟退火算法，保存最新的解
525 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
max_iter):
526     current_solution = initialize_solution()
527     # 确保满足豆类种植约束
528     current_solution = enforce_bean_constraint(current_solution)
529     current_solution = enforce_rotation_constraint(current_solution) # 新增重茬种
种植限制
530     current_solution = allocate_other_crops(current_solution) # 分配其他作物
531     current_objective = calculate_objective(current_solution)
532     best_solution = np.copy(current_solution)

```



```

533 latest_solution = np.copy(current_solution) # 保存最新解
534 best_objective = current_objective
535
536 temp = init_temp
537 objectives = [current_objective] # 存储每次迭代的目标函数值
538
539 iteration_count = 0 # 记录迭代次数
540
541 while temp > min_temp:
542     for _ in range(max_iter):
543         iteration_count += 1
544         new_solution = np.copy(current_solution)
545
546         # 随机调整一个地块某年的作物种植面积
547         land_index = random.randint(0, num_lands - 1)
548         year = random.randint(0, num_years - 1)
549         crop = random.choice(crop_list)
550         adjustment = round(random.uniform(-5, 5), 1)
551         new_solution[land_index, crop_list.index(crop), year] += adjustment
552
553         # 确保每个地块的总种植面积不超过地块面积
554         total_area_for_year = np.sum(new_solution[land_index, :, year])
555         if total_area_for_year > areas_C[land_index]:
556             # 调整作物面积以确保总面积不超过地块面积
557             new_solution[land_index, :, year] *= areas_C[land_index] /
total_area_for_year
558
559         # 确保满足豆类种植约束
560         new_solution = enforce_bean_constraint(new_solution)
561         # 确保避免连续重茬种植
562         new_solution = enforce_rotation_constraint(new_solution)
563
564         # 确保种植面积不为负
565         new_solution = enforce_non_negative_area(new_solution)
566
567         # 重新计算目标函数
568         new_objective = calculate_objective(new_solution)
569
570         # 接受新解的策略
571         if new_objective < current_objective:
572             current_solution = new_solution
573             current_objective = new_objective
574             if new_objective < best_objective:
575                 best_solution = np.copy(new_solution)
576                 best_objective = new_objective
577         else:
578             delta = new_objective - current_objective

```

```

579         if np.exp(-delta / temp) > random.uniform(0, 1):
580             current_solution = new_solution
581             current_objective = new_objective
582
583         # 保存最新的解
584         latest_solution = np.copy(current_solution)
585
586         objectives.append(current_objective) # 保存每次迭代后的目标函数值
587
588         temp *= cooling_rate
589
590     print(f"总迭代次数: {iteration_count}")
591     return best_solution, latest_solution, best_objective, objectives
592
593 # 可视化迭代过程中目标函数值的变化
594 def visualize_objective_iterations(objectives):
595     plt.figure(figsize=(5, 5))
596     plt.plot(objectives, label='Objective Value Over Iterations')
597     plt.xlabel('Iteration')
598     plt.ylabel('Objective Value')
599     plt.legend()
600     plt.grid(True)
601     plt.xticks([]) # 移除 x 轴上的刻度标签
602     plt.show()
603
604 # 保存最新的解到Excel文件
605 def save_solution_to_excel(solution, filename):
606     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
607         for year in range(num_years):
608             # 为每年创建一个sheet
609             sheet_name = f'Year_{years[year]}'
610             df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
611 index=[f'C{land+1}' for land in range(num_lands)])
612             df_solution.to_excel(writer, sheet_name=sheet_name)
613             print(f"最新解已保存到 {filename}")
614
615 # 参数设置
616 init_temp = 1000
617 min_temp = 1
618 cooling_rate = 0.98 # 放慢冷却速度
619 max_iter = 5000 # 增加迭代次数
620
621 # 运行模拟退火并获取每次迭代的目标函数值
622 best_solution, latest_solution, best_objective, objectives =
        simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
        max_iter)

```

```

623 # 可视化
624 visualize_objective_iterations(objectives)
625 #print(f"C最佳方案的作物种类数量之和: {latest_solution}")
626
627 # 调用保存函数，将最新解保存到文件
628 save_solution_to_excel(latest_solution, 'latest_solution_C.xlsx')
629
630 # 获取相关参数
631 years = df_D['Year'].values
632 crops = df_D.columns[1:]
633 num_years = len(years)
634 crop_list = list(crops)
635 planting_data = df_D.iloc[:, 1:].values # 提取各年份的种植数据
636 bean_crops = ['豇豆', '刀豆', '芸豆']
637 other_crops = [crop for crop in crop_list if crop not in bean_crops]
638
639 # D地块参数
640 areas_D = area_D['地块面积/亩'].tolist()
641 initial_bean_areas = area_D['2023年豆类种植面积'].tolist()
642 num_lands = len(areas_D)
643
644 # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
645 def initialize_solution():
646     solution = np.zeros((num_lands, len(crop_list), num_years))
647
648     # 先分配豆类作物
649     for land_index in range(num_lands):
650         for year in range(num_years):
651             # 保证豆类作物在三年内至少种植一次，且避免重茬种植
652             if year >= 2:
653                 total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
654                     land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
655                 if total_bean_area < areas_D[land_index]:
656                     # 随机选择豆类作物
657                     selected_bean = random.choice(bean_crops)
658                     available_area = areas_D[land_index] - np.sum(solution[
659                         land_index, :, year])
660                     if available_area > 0:
661                         solution[land_index, crop_list.index(selected_bean), year]
662                         = min(available_area, areas_D[land_index])
663
664     return solution
665
666 # 确保豆类作物满足三年种植约束（每次分配时都检验）
667 def enforce_bean_constraint(solution):
668     for land_index in range(num_lands):
669         for year in range(2, num_years): # 从第2年开始检查三年内的种植面积

```

```

667         total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
668         if total_bean_area < areas_D[land_index]:
669             # 需要调整豆类作物的种植面积
670             deficit = areas_D[land_index] - total_bean_area
671             selected_bean = random.choice(bean_crops)
672             selected_year = random.randint(year-2, year)
673             available_area = areas_D[land_index] - np.sum(solution[land_index,
:, selected_year])
674             if available_area > 0:
675                 solution[land_index, crop_list.index(selected_bean),
selected_year] += min(deficit, available_area)
676         return solution
677
678 # 避免连续重茬种植的约束
679 def enforce_rotation_constraint(solution):
680     for land_index in range(num_lands):
681         for year in range(1, num_years):
682             for crop_index in range(len(crop_list)):
683                 crop_area_year1 = solution[land_index, crop_index, year]
684                 crop_area_year0 = solution[land_index, crop_index, year - 1]
685                 if crop_area_year1 + crop_area_year0 > areas_D[land_index]: # 改为
areas_D
686                 adjustment = crop_area_year1 + crop_area_year0 - areas_D[
land_index]
687                 if crop_area_year1 > adjustment:
688                     solution[land_index, crop_index, year] -= adjustment
689                 else:
690                     solution[land_index, crop_index, year - 1] -= (adjustment -
crop_area_year1)
691         return solution
692
693 # 分配其他作物，避免重茬
694 def allocate_other_crops(solution):
695     for land_index in range(num_lands):
696         for year in range(num_years):
697             remaining_area = areas_D[land_index] - np.sum(solution[land_index, :,
year]) # 改为areas_D
698             if remaining_area > 0:
699                 selected_crop = random.choice(other_crops)
700                 solution[land_index, crop_list.index(selected_crop), year] = min(
remaining_area, areas_D[land_index])
701         return solution
702
703 # 确保种植面积不为负
704 def enforce_non_negative_area(solution):
705     for land_index in range(num_lands):

```

```

706         for year in range(num_years):
707             for crop_index in range(len(crop_list)):
708                 if solution[land_index, crop_index, year] < 0:
709                     solution[land_index, crop_index, year] = 0
710         return solution
711
712 # 计算目标函数：种植作物种类数量之和
713 def calculate_objective(solution):
714     total_crops = 0
715     for land_index in range(num_lands):
716         for year in range(num_years):
717             planted_crops = np.sum(solution[land_index, :, year] > 0)
718             total_crops += planted_crops
719     return total_crops
720
721 # 模拟退火算法
722 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
723     max_iter):
724     current_solution = initialize_solution()
725     current_solution = enforce_bean_constraint(current_solution)
726     current_solution = enforce_rotation_constraint(current_solution)
727     current_solution = allocate_other_crops(current_solution)
728     current_objective = calculate_objective(current_solution)
729     best_solution = np.copy(current_solution)
730     latest_solution = np.copy(current_solution)
731     best_objective = current_objective
732
733     temp = init_temp
734     objectives = [current_objective]
735     iteration_count = 0
736
737     while temp > min_temp:
738         for _ in range(max_iter):
739             iteration_count += 1
740             new_solution = np.copy(current_solution)
741             land_index = random.randint(0, num_lands - 1)
742             year = random.randint(0, num_years - 1)
743             crop = random.choice(crop_list)
744             adjustment = round(random.uniform(-5, 5), 1)
745             new_solution[land_index, crop_list.index(crop), year] += adjustment
746
747             total_area_for_year = np.sum(new_solution[land_index, :, year])
748             if total_area_for_year > areas_D[land_index]:
749                 new_solution[land_index, :, year] *= areas_D[land_index] /
750                 total_area_for_year
751
752             new_solution = enforce_bean_constraint(new_solution)

```

```

751         new_solution = enforce_rotation_constraint(new_solution)
752         new_solution = enforce_non_negative_area(new_solution)
753
754         new_objective = calculate_objective(new_solution)
755
756         if new_objective < current_objective:
757             current_solution = new_solution
758             current_objective = new_objective
759             if new_objective < best_objective:
760                 best_solution = np.copy(new_solution)
761                 best_objective = new_objective
762         else:
763             delta = new_objective - current_objective
764             if np.exp(-delta / temp) > random.uniform(0, 1):
765                 current_solution = new_solution
766                 current_objective = new_objective
767
768         latest_solution = np.copy(current_solution)
769         objectives.append(current_objective)
770
771         temp *= cooling_rate
772
773     print(f"总迭代次数: {iteration_count}")
774     return best_solution, latest_solution, best_objective, objectives
775
776 # 可视化
777 def visualize_objective_iterations(objectives):
778     plt.figure(figsize=(5, 5))
779     plt.plot(objectives, label='Objective Value Over Iterations')
780     plt.xlabel('Iteration')
781     plt.ylabel('Objective Value')
782     plt.legend()
783     plt.grid(True)
784     plt.xticks([])
785     plt.show()
786
787 # 保存解
788 def save_solution_to_excel(solution, filename):
789     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
790         for year in range(num_years):
791             sheet_name = f'Year_{years[year]}'
792             df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
793 index=[f'D{land+1}' for land in range(num_lands)]) # 改为D
794             df_solution.to_excel(writer, sheet_name=sheet_name)
795         print(f"最新解已保存到 {filename}")
796
797 # 参数

```

```

797 init_temp = 1000
798 min_temp = 1
799 cooling_rate = 0.98
800 max_iter = 5000
801
802 # 运行模拟退火算法
803 best_solution, latest_solution, best_objective, objectives =
    simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
    max_iter)
804
805 # 可视化
806 visualize_objective_iterations(objectives)
807
808 # 保存最新解
809 save_solution_to_excel(latest_solution, 'latest_solution_D.xlsx')
810
811 # 获取相关参数
812 years = df_E['Year'].values
813 crops = df_E.columns[1:]
814 num_years = len(years)
815 crop_list = list(crops)
816 planting_data = df_E.iloc[:, 1:].values # 提取各年份的种植数据
817 bean_crops = ['豇豆', '刀豆', '芸豆']
818 other_crops = [crop for crop in crop_list if crop not in bean_crops]
819
820 # E地块参数
821 areas_E = area_E['地块面积/亩'].tolist()
822 initial_bean_areas = area_E['2023年豆类种植面积'].tolist()
823 num_lands = len(areas_E)
824
825 # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
826 def initialize_solution():
827     solution = np.zeros((num_lands, len(crop_list), num_years))
828
829     # 先分配豆类作物
830     for land_index in range(num_lands):
831         for year in range(num_years):
832             # 保证豆类作物在三年内至少种植一次，且避免重茬种植
833             if year >= 2:
834                 total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
835                 if total_bean_area < areas_E[land_index]:
836                     # 随机选择豆类作物
837                     selected_bean = random.choice(bean_crops)
838                     available_area = areas_E[land_index] - np.sum(solution[
land_index, :, year])
839                     if available_area > 0:

```

```

840         solution[land_index, crop_list.index(selected_bean), year]
      = min(available_area, areas_E[land_index])
841
842     return solution
843
844 # 确保豆类作物满足三年种植约束（每次分配时都检验）
845 def enforce_bean_constraint(solution):
846     for land_index in range(num_lands):
847         for year in range(2, num_years): # 从第2年开始检查三年内的种植面积
848             total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
849             if total_bean_area < areas_E[land_index]:
850                 # 需要调整豆类作物的种植面积
851                 deficit = areas_E[land_index] - total_bean_area
852                 selected_bean = random.choice(bean_crops)
853                 selected_year = random.randint(year-2, year)
854                 available_area = areas_E[land_index] - np.sum(solution[land_index,
:, selected_year])
855                 if available_area > 0:
856                     solution[land_index, crop_list.index(selected_bean),
selected_year] += min(deficit, available_area)
857     return solution
858
859 # 避免连续重茬种植的约束
860 def enforce_rotation_constraint(solution):
861     for land_index in range(num_lands):
862         for year in range(1, num_years):
863             for crop_index in range(len(crop_list)):
864                 crop_area_year1 = solution[land_index, crop_index, year]
865                 crop_area_year0 = solution[land_index, crop_index, year - 1]
866                 if crop_area_year1 + crop_area_year0 > areas_E[land_index]: # 改为
areas_E
867                 adjustment = crop_area_year1 + crop_area_year0 - areas_E[
land_index]
868                 if crop_area_year1 > adjustment:
869                     solution[land_index, crop_index, year] -= adjustment
870                 else:
871                     solution[land_index, crop_index, year - 1] -= (adjustment -
crop_area_year1)
872     return solution
873
874 # 分配其他作物，避免重茬
875 def allocate_other_crops(solution):
876     for land_index in range(num_lands):
877         for year in range(num_years):
878             remaining_area = areas_E[land_index] - np.sum(solution[land_index, :,
year]) # 改为areas_E

```



```

879         if remaining_area > 0:
880             selected_crop = random.choice(other_crops)
881             solution[land_index, crop_list.index(selected_crop), year] = min(
remaining_area, areas_E[land_index])
882         return solution
883
884 # 确保种植面积不为负
885 def enforce_non_negative_area(solution):
886     for land_index in range(num_lands):
887         for year in range(num_years):
888             for crop_index in range(len(crop_list)):
889                 if solution[land_index, crop_index, year] < 0:
890                     solution[land_index, crop_index, year] = 0
891     return solution
892
893 # 计算目标函数：种植作物种类数量之和
894 def calculate_objective(solution):
895     total_crops = 0
896     for land_index in range(num_lands):
897         for year in range(num_years):
898             planted_crops = np.sum(solution[land_index, :, year] > 0)
899             total_crops += planted_crops
900     return total_crops
901
902 # 模拟退火算法
903 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
max_iter):
904     current_solution = initialize_solution()
905     current_solution = enforce_bean_constraint(current_solution)
906     current_solution = enforce_rotation_constraint(current_solution)
907     current_solution = allocate_other_crops(current_solution)
908     current_objective = calculate_objective(current_solution)
909     best_solution = np.copy(current_solution)
910     latest_solution = np.copy(current_solution)
911     best_objective = current_objective
912
913     temp = init_temp
914     objectives = [current_objective]
915     iteration_count = 0
916
917     while temp > min_temp:
918         for _ in range(max_iter):
919             iteration_count += 1
920             new_solution = np.copy(current_solution)
921             land_index = random.randint(0, num_lands - 1)
922             year = random.randint(0, num_years - 1)
923             crop = random.choice(crop_list)

```

```

924         adjustment = round(random.uniform(-5, 5), 1)
925         new_solution[land_index, crop_list.index(crop), year] += adjustment
926
927         total_area_for_year = np.sum(new_solution[land_index, :, year])
928         if total_area_for_year > areas_E[land_index]:
929             new_solution[land_index, :, year] *= areas_E[land_index] /
total_area_for_year
930
931         new_solution = enforce_bean_constraint(new_solution)
932         new_solution = enforce_rotation_constraint(new_solution)
933         new_solution = enforce_non_negative_area(new_solution)
934
935         new_objective = calculate_objective(new_solution)
936
937         if new_objective < current_objective:
938             current_solution = new_solution
939             current_objective = new_objective
940             if new_objective < best_objective:
941                 best_solution = np.copy(new_solution)
942                 best_objective = new_objective
943         else:
944             delta = new_objective - current_objective
945             if np.exp(-delta / temp) > random.uniform(0, 1):
946                 current_solution = new_solution
947                 current_objective = new_objective
948
949         latest_solution = np.copy(current_solution)
950         objectives.append(current_objective)
951
952         temp *= cooling_rate
953
954         print(f"总迭代次数: {iteration_count}")
955         return best_solution, latest_solution, best_objective, objectives
956
957 # 可视化
958 def visualize_objective_iterations(objectives):
959     plt.figure(figsize=(5, 5))
960     plt.plot(objectives, label='Objective Value Over Iterations')
961     plt.xlabel('Iteration')
962     plt.ylabel('Objective Value')
963     plt.legend()
964     plt.grid(True)
965     plt.xticks([])
966     plt.show()
967
968 # 保存解
969 def save_solution_to_excel(solution, filename):

```

```

970     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
971         for year in range(num_years):
972             sheet_name = f'Year_{years[year]}'
973             df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
index=[f'E{land+1}' for land in range(num_lands)]) # 改为E
974             df_solution.to_excel(writer, sheet_name=sheet_name)
975             print(f"最新解已保存到 {filename}")
976
977 # 参数
978 init_temp = 1000
979 min_temp = 1
980 cooling_rate = 0.98
981 max_iter = 5000
982
983 # 运行模拟退火算法
984 best_solution, latest_solution, best_objective, objectives =
simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
max_iter)
985
986 # 可视化
987 visualize_objective_iterations(objectives)
988
989 # 保存最新解
990 save_solution_to_excel(latest_solution, 'latest_solution_E.xlsx')
991
992 # 获取相关参数
993 years = df_F['Year'].values
994 crops = df_F.columns[1:]
995 num_years = len(years)
996 crop_list = list(crops)
997 planting_data = df_F.iloc[:, 1:].values # 提取各年份的种植数据
998 bean_crops = ['豇豆', '刀豆', '芸豆']
999 other_crops = [crop for crop in crop_list if crop not in bean_crops]
1000
1001 # F地块参数
1002 areas_F = area_F['地块面积/亩'].tolist()
1003 initial_bean_areas = area_F['2023年豆类种植面积'].tolist()
1004 num_lands = len(areas_F)
1005
1006 # 初始化种植规划方案，先分配豆类作物，满足重茬和三年种植约束
1007 def initialize_solution():
1008     solution = np.zeros((num_lands, len(crop_list), num_years))
1009
1010     # 先分配豆类作物
1011     for land_index in range(num_lands):
1012         for year in range(num_years):
1013             # 保证豆类作物在三年内至少种植一次，且避免重茬种植

```

```

1014         if year >= 2:
1015             total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
1016             if total_bean_area < areas_F[land_index]:
1017                 # 随机选择豆类作物
1018                 selected_bean = random.choice(bean_crops)
1019                 available_area = areas_F[land_index] - np.sum(solution[
land_index, :, year])
1020                 if available_area > 0:
1021                     solution[land_index, crop_list.index(selected_bean), year]
= min(available_area, areas_F[land_index])
1022
1023     return solution
1024
1025 # 确保豆类作物满足三年种植约束（每次分配时都检验）
1026 def enforce_bean_constraint(solution):
1027     for land_index in range(num_lands):
1028         for year in range(2, num_years): # 从第2年开始检查三年内的种植面积
1029             total_bean_area = initial_bean_areas[land_index] + np.sum(solution[
land_index, [crop_list.index(bean) for bean in bean_crops], year-2:year+1])
1030             if total_bean_area < areas_F[land_index]:
1031                 # 需要调整豆类作物的种植面积
1032                 deficit = areas_F[land_index] - total_bean_area
1033                 selected_bean = random.choice(bean_crops)
1034                 selected_year = random.randint(year-2, year)
1035                 available_area = areas_F[land_index] - np.sum(solution[land_index,
:, selected_year])
1036                 if available_area > 0:
1037                     solution[land_index, crop_list.index(selected_bean),
selected_year] += min(deficit, available_area)
1038     return solution
1039
1040 # 避免连续重茬种植的约束
1041 def enforce_rotation_constraint(solution):
1042     for land_index in range(num_lands):
1043         for year in range(1, num_years):
1044             for crop_index in range(len(crop_list)):
1045                 crop_area_year1 = solution[land_index, crop_index, year]
1046                 crop_area_year0 = solution[land_index, crop_index, year - 1]
1047                 if crop_area_year1 + crop_area_year0 > areas_F[land_index]: # 改为
areas_F
1048                 adjustment = crop_area_year1 + crop_area_year0 - areas_F[
land_index]
1049                 if crop_area_year1 > adjustment:
1050                     solution[land_index, crop_index, year] -= adjustment
1051                 else:
1052                     solution[land_index, crop_index, year - 1] -= (adjustment -

```

```

        crop_area_year1)
1053     return solution
1054
1055 # 分配其他作物，避免重茬
1056 def allocate_other_crops(solution):
1057     for land_index in range(num_lands):
1058         for year in range(num_years):
1059             remaining_area = areas_F[land_index] - np.sum(solution[land_index, :,
1060 year]) # 改为areas_F
1061             if remaining_area > 0:
1062                 selected_crop = random.choice(other_crops)
1063                 solution[land_index, crop_list.index(selected_crop), year] = min(
1064 remaining_area, areas_F[land_index])
1065     return solution
1066
1067 # 确保种植面积不为负
1068 def enforce_non_negative_area(solution):
1069     for land_index in range(num_lands):
1070         for year in range(num_years):
1071             for crop_index in range(len(crop_list)):
1072                 if solution[land_index, crop_index, year] < 0:
1073                     solution[land_index, crop_index, year] = 0
1074     return solution
1075
1076 # 计算目标函数：种植作物种类数量之和
1077 def calculate_objective(solution):
1078     total_crops = 0
1079     for land_index in range(num_lands):
1080         for year in range(num_years):
1081             planted_crops = np.sum(solution[land_index, :, year] > 0)
1082             total_crops += planted_crops
1083     return total_crops
1084
1085 # 模拟退火算法
1086 def simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
1087 max_iter):
1088     current_solution = initialize_solution()
1089     current_solution = enforce_bean_constraint(current_solution)
1090     current_solution = enforce_rotation_constraint(current_solution)
1091     current_solution = allocate_other_crops(current_solution)
1092     current_objective = calculate_objective(current_solution)
1093     best_solution = np.copy(current_solution)
1094     latest_solution = np.copy(current_solution)
1095     best_objective = current_objective
1096
1097     temp = init_temp
1098     objectives = [current_objective]

```

```

1096     iteration_count = 0
1097
1098     while temp > min_temp:
1099         for _ in range(max_iter):
1100             iteration_count += 1
1101             new_solution = np.copy(current_solution)
1102             land_index = random.randint(0, num_lands - 1)
1103             year = random.randint(0, num_years - 1)
1104             crop = random.choice(crop_list)
1105             adjustment = round(random.uniform(-5, 5), 1)
1106             new_solution[land_index, crop_list.index(crop), year] += adjustment
1107
1108             total_area_for_year = np.sum(new_solution[land_index, :, year])
1109             if total_area_for_year > areas_F[land_index]:
1110                 new_solution[land_index, :, year] *= areas_F[land_index] /
total_area_for_year
1111
1112             new_solution = enforce_bean_constraint(new_solution)
1113             new_solution = enforce_rotation_constraint(new_solution)
1114             new_solution = enforce_non_negative_area(new_solution)
1115
1116             new_objective = calculate_objective(new_solution)
1117
1118             if new_objective < current_objective:
1119                 current_solution = new_solution
1120                 current_objective = new_objective
1121                 if new_objective < best_objective:
1122                     best_solution = np.copy(new_solution)
1123                     best_objective = new_objective
1124             else:
1125                 delta = new_objective - current_objective
1126                 if np.exp(-delta / temp) > random.uniform(0, 1):
1127                     current_solution = new_solution
1128                     current_objective = new_objective
1129
1130             latest_solution = np.copy(current_solution)
1131             objectives.append(current_objective)
1132
1133             temp *= cooling_rate
1134
1135     print(f"总迭代次数: {iteration_count}")
1136     return best_solution, latest_solution, best_objective, objectives
1137
1138 # 可视化
1139 def visualize_objective_iterations(objectives):
1140     plt.figure(figsize=(5, 5))
1141     plt.plot(objectives, label='Objective Value Over Iterations')

```

```

1142     plt.xlabel('Iteration')
1143     plt.ylabel('Objective Value')
1144     plt.legend()
1145     plt.grid(True)
1146     plt.xticks([])
1147     plt.show()
1148
1149 # 保存解
1150 def save_solution_to_excel(solution, filename):
1151     with pd.ExcelWriter(filename, engine='xlsxwriter') as writer:
1152         for year in range(num_years):
1153             sheet_name = f'Year_{years[year]}'
1154             df_solution = pd.DataFrame(solution[:, :, year], columns=crop_list,
1155                                     index=[f'F{land+1}' for land in range(num_lands)]) # 改为F
1156             df_solution.to_excel(writer, sheet_name=sheet_name)
1157             print(f"最新解已保存到 {filename}")
1158
1159 # 参数
1160 init_temp = 1000
1161 min_temp = 1
1162 cooling_rate = 0.98
1163 max_iter = 5000
1164
1165 # 运行模拟退火算法
1166 best_solution, latest_solution, best_objective, objectives =
1167     simulated_annealing_with_visualization(init_temp, min_temp, cooling_rate,
1168     max_iter)
1169
1170 # 可视化
1171 visualize_objective_iterations(objectives)
1172
1173 # 保存最新解
1174 save_solution_to_excel(latest_solution, 'latest_solution_F.xlsx')

```

3. qn_3_ 填入 result.py

```

1  # -*- coding: utf-8 -*-
2
3  # -*- coding: utf-8 -*-
4  import pandas as pd
5  from openpyxl import load_workbook
6
7  # 加载所有数据源文件
8  source_files = {
9      'A': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
10         latest_solution_A.xlsx', sheet_name=None),
11      'B': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
12         latest_solution_B.xlsx', sheet_name=None),

```

```

11     'C': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
latest_solution_C.xlsx', sheet_name=None),
12     'D': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
latest_solution_D.xlsx', sheet_name=None),
13     'E': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
latest_solution_E.xlsx', sheet_name=None),
14     'F': pd.read_excel('C:/Users/Administrator/Desktop/latest/问题二/
latest_solution_F.xlsx', sheet_name=None)
15 }
16
17 # 加载目标文件
18 target_file_path = 'C:/Users/Administrator/Desktop/C题/附件3/mm.xlsx'
19 wb = load_workbook(target_file_path)
20 sheet_names = wb.sheetnames
21
22 # 需要填写的地块编号（确保地块与目标文件一致）
23 blocks_to_fill = [
24     'A1', 'A2', 'A3', 'A4', 'A5', 'A6',
25     'B1', 'B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B9', 'B10', 'B11', 'B12', 'B13',
26     'B14',
27     'C1', 'C2', 'C3', 'C4', 'C5', 'C6',
28     'D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8',
29     'E1', 'E2', 'E3', 'E4', 'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13',
30     'E14', 'E15', 'E16',
31     'F1', 'F2', 'F3', 'F4'
32 ]
33
34 # 定义作物匹配逻辑，逐行逐列匹配
35 def match_and_fill_data(source_sheet, target_sheet, season_rows, season_crops,
36                          block_list):
37     target_crops = [target_sheet.cell(row=1, column=col_idx).value for col_idx in
38                     range(3, 44)] # 第3列到第43列为作物种类
39     target_blocks = [target_sheet.cell(row=row_idx, column=2).value for row_idx in
40                      season_rows]
41
42     for row_idx, block in enumerate(target_blocks):
43         if block in block_list:
44             for col_idx, crop in enumerate(target_crops, start=3):
45                 target_cell = target_sheet.cell(row=row_idx + season_rows.start,
46                                                  column=col_idx)
47                 if target_cell.value is None:
48                     if crop in source_sheet.columns and block in source_sheet.iloc
49                       [:, 0].values:
50                         source_row = source_sheet[source_sheet.iloc[:, 0] == block
51                        ].index[0]
52                         source_value = source_sheet.iloc[source_row, source_sheet.
53                        columns.get_loc(crop)]

```



```

45         target_cell.value = source_value
46         print(f"填充数据：地块 {block} 作物 {crop}, 值 {
source_value}, 目标单元格 {target_cell.coordinate}")
47
48 # 遍历每个sheet，按季节填充相应的数据
49 for i, sheet_name in enumerate(sheet_names):
50     target_sheet = wb[sheet_name]
51
52     source_sheet_a = source_files['A'][list(source_files['A'].keys())[i]]
53     source_sheet_b = source_files['B'][list(source_files['B'].keys())[i]]
54     source_sheet_c = source_files['C'][list(source_files['C'].keys())[i]]
55     source_sheet_d = source_files['D'][list(source_files['D'].keys())[i]]
56     source_sheet_e = source_files['E'][list(source_files['E'].keys())[i]]
57     source_sheet_f = source_files['F'][list(source_files['F'].keys())[i]]
58
59     # 第一季作物分类：ABC 文件中的所有作物
60     first_season_crops_abc = list(source_sheet_a.columns)[1:] # ABC 文件中的所有作
作物归类为第一季
61     first_season_crops_d = list(source_sheet_d.columns)[1:] # D 文件中的所有作物归
类为第一季
62     first_season_crops_e = list(source_sheet_e.columns)[1:] # E 文件中的所有作物归
类为第一季
63     first_season_crops_f = list(source_sheet_f.columns)[1:] # F 文件中的所有作物归
类为第一季
64
65     # 第一季数据填充（第2行到第55行，第3列到第43列）
66     match_and_fill_data(source_sheet_a, target_sheet, range(1, 56),
first_season_crops_abc, blocks_to_fill)
67     match_and_fill_data(source_sheet_b, target_sheet, range(1, 56),
first_season_crops_abc, blocks_to_fill)
68     match_and_fill_data(source_sheet_c, target_sheet, range(1, 56),
first_season_crops_abc, blocks_to_fill)
69     match_and_fill_data(source_sheet_d, target_sheet, range(1, 56),
first_season_crops_d, blocks_to_fill)
70     match_and_fill_data(source_sheet_e, target_sheet, range(1, 56),
first_season_crops_e, blocks_to_fill)
71     match_and_fill_data(source_sheet_f, target_sheet, range(1, 56),
first_season_crops_f, blocks_to_fill)
72
73 # 保存文件
74 output_file = 'C:/Users/Administrator/Desktop/result2-v4.xlsx'
75 wb.save(output_file)
76
77 print(f"数据已成功保存至：{output_file}")

```

4. q2_0.py

```

1 # -*- coding: utf-8 -*-

```

```

2
3 import numpy as np
4 import pandas as pd
5
6 # 读取Excel文件的Sheet1
7 file_path = 'C:/Users/86188/Desktop/2024数模/问题一/Q1.xlsx'
8 data = pd.read_excel(file_path, sheet_name=0)
9
10 # 读取sigma_P^2的文件
11 sigma_p_file_path = 'C:/Users/86188/Desktop/2024数模/问题二/sigma_p.xlsx'
12 sigma_p_data = pd.read_excel(sigma_p_file_path)
13
14 # 将方差与作物名称映射
15 sigma_p_dict = dict(zip(sigma_p_data['Unnamed: 0'], sigma_p_data['方差']))
16
17 # 定义蔬菜类和食用菌类作物
18 vegetables = ['豇豆', '刀豆', '芸豆', '土豆', '西红柿', '茄子', '菠菜',
19               '青椒', '菜花', '包菜', '油麦菜', '小青菜', '黄瓜', '生菜',
20               '辣椒', '空心菜', '黄心菜', '芹菜', '大白菜', '白萝卜', '红萝卜']
21 mushrooms = ['榆黄菇', '香菇', '白灵菇']
22
23 # 更新需求量的函数
24 def update_demand(original_demand, sigma_D):
25     uniform_random = np.random.uniform(0.05, 0.10)
26     normal_random = np.random.normal(0, sigma_D)
27     return original_demand * (1 + uniform_random + normal_random)
28
29 # 更新成本的函数
30 def update_cost(original_cost, sigma_c):
31     normal_random = np.random.normal(0, sigma_c)
32     return original_cost * (1.05 + normal_random)
33
34 # 更新蔬菜类单价的函数
35 def update_price(original_price, sigma_p):
36     normal_random = np.random.normal(0, np.sqrt(sigma_p))
37     return original_price * (1.05 + normal_random)
38
39 # 更新食用菌类单价的函数
40 def update_mushroom_price(original_price, sigma_p):
41     uniform_random = np.random.uniform(-0.05, -0.01)
42     normal_random = np.random.normal(0, np.sqrt(sigma_p))
43     return original_price * (1 + uniform_random + normal_random)
44
45 # 更新羊肚菌单价的函数
46 def update_yangdujun_price(original_price, sigma_p):
47     normal_random = np.random.normal(0, np.sqrt(sigma_p))
48     return original_price * (0.95 + normal_random)

```

```

49
50 # 对应小麦和玉米的sigma_D
51 sigma_D_dict = {'小麦': 0.005, '玉米': 0.010}
52
53 # 对应所有成本的sigma_c
54 sigma_c = 0.5
55
56 # 创建一个字典，用于存储7年的数据
57 yearly_data = {}
58
59 # 进行7年的数据更新
60 for year in range(1, 8):
61     # 第一年的数据从初始数据开始，后面的年份基于上一年
62     if year == 1:
63         yearly_data[year] = data.copy()
64     else:
65         yearly_data[year] = yearly_data[year-1].copy()
66
67     # 更新小麦和玉米的需求量
68     for crop in ['小麦', '玉米']:
69         sigma_D = sigma_D_dict[crop]
70         yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '需求量'] =
yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '需求量'].apply(
lambda x: update_demand(x, sigma_D))
71
72     # 更新成本
73     cost_columns = ['平旱地成本', '梯田成本', '山坡地成本', '水浇地成本', '普通大棚
成本', '智慧大棚成本']
74     for col in cost_columns:
75         yearly_data[year][col] = yearly_data[year][col].apply(lambda x: update_cost
(x, sigma_c))
76
77     # 更新蔬菜类的非智慧单价和智慧单价
78     for crop in vegetables:
79         if crop in sigma_p_dict:
80             sigma_p = sigma_p_dict[crop]
81             yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '非智慧单
价'] = yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '非智慧单价'
].apply(lambda x: update_price(x, sigma_p))
82             yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '智慧单价'
] = yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '智慧单价'].
apply(lambda x: update_price(x, sigma_p))
83
84     # 更新食用菌类的非智慧单价和智慧单价
85     for crop in mushrooms:
86         if crop in sigma_p_dict:
87             sigma_p = sigma_p_dict[crop]

```

```

88         yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '非智慧单
            价'] = yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '非智慧单
            价'].apply(lambda x: update_mushroom_price(x, sigma_p))
89         yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '智慧单
            价'] = yearly_data[year].loc[yearly_data[year]['作物名称'] == crop, '智慧单
            价'].apply(lambda x: update_mushroom_price(x, sigma_p))
90
91     # 更新羊肚菌的非智慧单价和智慧单价
92     if '羊肚菌' in sigma_p_dict:
93         sigma_p = sigma_p_dict['羊肚菌']
94         yearly_data[year].loc[yearly_data[year]['作物名称'] == '羊肚菌', '非智慧单
            价'] = yearly_data[year].loc[yearly_data[year]['作物名称'] == '羊肚菌', '非智慧
            单价'].apply(lambda x: update_yangdujun_price(x, sigma_p))
95         yearly_data[year].loc[yearly_data[year]['作物名称'] == '羊肚菌', '智慧单
            价'] = yearly_data[year].loc[yearly_data[year]['作物名称'] == '羊肚菌', '智慧单
            价'].apply(lambda x: update_yangdujun_price(x, sigma_p))
96
97     # 将7年的数据保存到一个Excel文件的不同sheet中
98     with pd.ExcelWriter('C:/Users/86188/Desktop/2024数模/问题二/七年数据.xlsx') as
        writer:
99         for year in range(1, 8):
100             yearly_data[year].to_excel(writer, sheet_name=f'Year_{year}', index=False)
101
102     import numpy as np
103     import pandas as pd
104
105     # 读取七年数据的Excel文件
106     file_path = 'C:/Users/86188/Desktop/2024数模/问题二/七年数据.xlsx'
107     excel_data = pd.read_excel(file_path, sheet_name=None) # 读取所有sheet
108
109     # 读取sigma_p
110     sigma_p_file_path = 'C:/Users/86188/Desktop/2024数模/问题二/sigma_p.xlsx'
111     sigma_p_data = pd.read_excel(sigma_p_file_path)
112
113     # 将方差与作物名称映射
114     sigma_p_dict = dict(zip(sigma_p_data['Unnamed: 0'], sigma_p_data['方差']))
115
116     # 定义粮食作物类作物
117     grain_crops = [
118         '黄豆', '黑豆', '红豆', '绿豆', '爬豆', '小麦', '玉米', '谷子',
119         '高粱', '黍子', '荞麦', '南瓜', '红薯', '莜麦', '大麦', '水稻'
120     ]
121
122     # 定义函数用于更新需求量（除了玉米和小麦）
123     def update_other_crops_demand(original_demand):
124         normal_random = np.random.normal(0, 0.05) # N(0, 0.05)
125         return original_demand * (1 + normal_random)

```

```

126
127 # 定义函数用于更新产量
128 def update_yield(original_yield):
129     normal_random = np.random.normal(0, 0.10) # N(0, 0.10)
130     return original_yield * (1 + normal_random)
131
132 # 定义函数用于更新单价
133 def update_price(original_price, sigma_p):
134     normal_random = np.random.normal(0, sigma_p)
135     return original_price * (1 + normal_random)
136
137 # 对每个sheet进行处理
138 for sheet_name, sheet_data in excel_data.items():
139     # 过滤出不是玉米和小麦的作物
140     other_crops = sheet_data[~sheet_data['作物名称'].isin(['玉米', '小麦'])]
141
142     # 更新需求量
143     sheet_data.loc[~sheet_data['作物名称'].isin(['玉米', '小麦']), '需求量'] =
other_crops['需求量'].apply(update_other_crops_demand)
144
145     # 保存处理后的数据到原sheet
146     excel_data[sheet_name] = sheet_data
147
148 # 需要更新的产量列
149 yield_columns = ['平旱地产量', '山坡地产量', '梯田产量', '水浇地产量', '普通大棚产
量', '智慧大棚产量']
150
151 # 对每个sheet进行处理
152 for sheet_name, sheet_data in excel_data.items():
153     # 对每个需要更新的产量列应用公式
154     for col in yield_columns:
155         if col in sheet_data.columns: # 检查列是否存在
156             sheet_data[col] = sheet_data[col].apply(update_yield)
157
158     # 保存处理后的数据到原sheet
159     excel_data[sheet_name] = sheet_data
160
161 # 对每个sheet进行处理
162 for sheet_name, sheet_data in excel_data.items():
163     # 筛选出属于粮食作物类的行
164     grain_crop_data = sheet_data[sheet_data['作物名称'].isin(grain_crops)]
165
166     # 更新智慧单价和非智慧单价
167     for crop in grain_crops:
168         if crop in sigma_p_dict:
169             sigma_p = sigma_p_dict[crop]
170             # 更新非智慧单价

```

```

171         sheet_data.loc[sheet_data['作物名称'] == crop, '非智慧单价'] =
grain_crop_data.loc[grain_crop_data['作物名称'] == crop, '非智慧单价'].apply(
lambda x: update_price(x, sigma_p))
172         # 更新智慧单价
173         sheet_data.loc[sheet_data['作物名称'] == crop, '智慧单价'] =
grain_crop_data.loc[grain_crop_data['作物名称'] == crop, '智慧单价'].apply(
lambda x: update_price(x, sigma_p))
174
175         # 保存处理后的数据到原sheet
176         excel_data[sheet_name] = sheet_data
177
178     # 创建新sheet名称, 按年份2024年到2030年
179     years = [f'{year}年' for year in range(2024, 2031)]
180
181     # 保存处理后的数据到一个新的Excel文件, 并重命名sheet
182     output_file_path = 'C:/Users/86188/Desktop/2024数模/问题二/七年数据_修改后.xlsx'
183     with pd.ExcelWriter(output_file_path) as writer:
184         for sheet_name, new_sheet_name in zip(excel_data.keys(), years):
185             excel_data[sheet_name].to_excel(writer, sheet_name=new_sheet_name, index=
False)

```

5. q2_2.py

```

1  # -*- coding: utf-8 -*-
2
3  import pandas as pd
4  import numpy as np
5  import random
6  import matplotlib.pyplot as plt
7
8  # 基础数据初始化
9  num_years = 7 # 2024——2030
10 # ABC: 粮食类
11 areas = {'A': 385, 'B': 619, 'C': 108} # ABC土地类型的面积
12 num_lands_ABC = 3 # ABC是粮食类
13 n1_A = 140 # A: 第0年豆类种植量 (单位: 亩)
14 n1_B = 199 # B: 第0年豆类种植量 (单位: 亩)
15 n1_C = 35 # C: 第0年豆类种植量 (单位: 亩)
16
17 # DEF: 蔬菜类
18 areas_DEF = {'E': 9.6, 'F': 2.4} # E、F地块面积固定
19 num_lands_DEF = 3 # DEF是蔬菜类地块
20 n1_D = 22 # D: 第0年豆类种植量 (单位: 亩)
21 n1_E = 3.6 # E: 第0年豆类种植量 (单位: 亩)
22 n1_F = 1.2 # F: 第0年豆类种植量 (单位: 亩)
23
24 # 读取Excel文件列表
25 file_list = ['C:/Users/86188/Desktop/2024数模/问题二/crop1.xlsx',

```

```

26         'C:/Users/86188/Desktop/2024数模/问题二/crop2.xlsx',
27         'C:/Users/86188/Desktop/2024数模/问题二/crop3.xlsx',
28         'C:/Users/86188/Desktop/2024数模/问题二/crop4.xlsx',
29         'C:/Users/86188/Desktop/2024数模/问题二/crop5.xlsx',
30         'C:/Users/86188/Desktop/2024数模/问题二/crop6.xlsx',
31         'C:/Users/86188/Desktop/2024数模/问题二/crop7.xlsx',
32         'C:/Users/86188/Desktop/2024数模/问题二/crop8.xlsx',
33         'C:/Users/86188/Desktop/2024数模/问题二/crop9.xlsx',
34         'C:/Users/86188/Desktop/2024数模/问题二/crop10.xlsx'
35     ]# 使用多个文件来计算平均利润
36
37 # 动态更新每年的需求量、单价、产量、成本
38 def read_crop_data(file_list):
39     crop_demand_by_year = {}
40     crop_prices_by_year = {}
41     crop_price_f_by_year = {}
42     crop_yield_by_land_by_year = {}
43     crop_cost_by_land_by_year = {}
44
45     for file in file_list:
46         crop_data = {}
47         for year in range(2024, 2031): # 2024-2030共七年
48             sheet_name = f'{year}年'
49             crop_data[year] = pd.read_excel(file, sheet_name=sheet_name)
50
51             # 剔除不需要的作物
52             crops_to_remove = ['水稻', '大白菜', '红萝卜', '白萝卜', '榆黄菇', '香菇',
53                                '白灵菇', '羊肚菌']
54             for year in range(2024, 2031):
55                 crop_data[year] = crop_data[year][~crop_data[year]['作物名称'].isin(
56                     crops_to_remove)]
57
58             crop_demand_by_year[file] = {year: crop_data[year]['需求量'].values for
59 year in range(2024, 2031)}
60             crop_prices_by_year[file] = {year: crop_data[year]['非智慧单价'].values for
61 year in range(2024, 2031)}
62             crop_price_f_by_year[file] = {year: crop_data[year]['智慧单价'].values for
63 year in range(2024, 2031)}
64
65             crop_yield_by_land_by_year[file] = {
66                 year: {
67                     'A': crop_data[year]['平旱地产量'].values[:15],
68                     'B': crop_data[year]['梯田产量'].values[:15],
69                     'C': crop_data[year]['山坡地产量'].values[:15],
70                     'D': crop_data[year]['水浇地产量'].values[15:33],
71                     'E': crop_data[year]['普通大棚产量'].values[15:33],
72                     'F': crop_data[year]['智慧大棚产量'].values[15:33],

```

```

68         }
69         for year in range(2024, 2031)
70     }
71
72     crop_cost_by_land_by_year[file] = {
73         year: {
74             'A': crop_data[year]['平旱地成本'].values[:15],
75             'B': crop_data[year]['梯田成本'].values[:15],
76             'C': crop_data[year]['山坡地成本'].values[:15],
77             'D': crop_data[year]['水浇地成本'].values[15:33],
78             'E': crop_data[year]['普通大棚成本'].values[15:33],
79             'F': crop_data[year]['智慧大棚成本'].values[15:33],
80         }
81         for year in range(2024, 2031)
82     }
83
84     return crop_demand_by_year, crop_prices_by_year, crop_price_f_by_year,
85         crop_yield_by_land_by_year, crop_cost_by_land_by_year
86
87 # 计算当年D地块的面积
88 def calculate_dynamic_D_area(file, year, crop_demand_by_year,
89     crop_yield_by_land_by_year):
90     """计算D地块面积: 139 - 当年水稻需求量 / 当年水浇地产量"""
91     rice_demand = crop_demand_by_year[file][year][0] # 水稻的需求量
92     rice_yield = crop_yield_by_land_by_year[file][year]['D'][0] # 水稻在水浇地的产
93     量
94     if rice_yield > 0: # 避免除以0
95         return 139 - rice_demand / rice_yield
96     else:
97         return 139 # 如果产量为0, 默认返回139
98
99 # 修改目标函数: 计算多个文件下的平均利润
100 def calculate_average_profit(solution, file_list, crop_demand_by_year,
101     crop_prices_by_year, crop_yield_by_land_by_year, crop_cost_by_land_by_year,
102     calculate_profit_func):
103     """计算多个文件下的平均利润"""
104     total_profit = 0
105     for file in file_list:
106         total_profit += calculate_profit_func(solution, file, crop_demand_by_year,
107             crop_prices_by_year, crop_yield_by_land_by_year, crop_cost_by_land_by_year)
108
109     # 返回平均利润
110     return total_profit / len(file_list)
111
112 # 计算ABC的利润
113 def calculate_profit_ABC(solution, file, crop_demand_by_year, crop_prices_by_year,
114     crop_yield_by_land_by_year, crop_cost_by_land_by_year):

```



```

108     """计算ABC粮食类作物的总利润"""
109     total_profit = 0
110     for land_index, land_name in enumerate(['A', 'B', 'C']):
111         for crop in range(15):
112             for year_index, year in enumerate(range(2024, 2031)):
113                 total_yield = solution[land_index, crop, year_index] *
crop_yield_by_land_by_year[file][year][land_name][crop]
114                 total_cost = solution[land_index, crop, year_index] *
crop_cost_by_land_by_year[file][year][land_name][crop]
115
116                 total_yield = round(total_yield, 1)
117                 total_cost = round(total_cost, 1)
118
119                 if total_yield > crop_demand_by_year[file][year][crop]:
120                     regular_revenue = crop_demand_by_year[file][year][crop] *
crop_prices_by_year[file][year][crop]
121                     excess_yield = total_yield - crop_demand_by_year[file][year][
crop]
122                     discount_revenue = excess_yield * (crop_prices_by_year[file][
year][crop] * 0.5)
123                     revenue = regular_revenue + discount_revenue
124                 else:
125                     revenue = total_yield * crop_prices_by_year[file][year][crop]
126
127                 total_profit += round(revenue - total_cost, 1)
128     return total_profit
129
130 # 计算DEF的利润
131 def calculate_profit_DEF(solution, file, crop_demand_by_year, crop_prices_by_year,
crop_yield_by_land_by_year, crop_cost_by_land_by_year):
132     """计算DEF蔬菜类作物的总利润"""
133     total_profit = 0
134     for land_index, land_name in enumerate(['D', 'E', 'F']):
135         for crop in range(18): # 确保作物范围在0-17
136             for year_index, year in enumerate(range(2024, 2031)):
137                 total_yield = solution[land_index, crop, year_index] *
crop_yield_by_land_by_year[file][year][land_name][crop]
138                 total_cost = solution[land_index, crop, year_index] *
crop_cost_by_land_by_year[file][year][land_name][crop]
139
140                 if land_name == 'F':
141                     price = crop_prices_by_year[file][year][crop]
142                 else:
143                     price = crop_prices_by_year[file][year][crop + 15]
144
145                 if total_yield > crop_demand_by_year[file][year][crop + 15]:
146                     regular_revenue = crop_demand_by_year[file][year][crop + 15] *

```

```

147         price
148             excess_yield = total_yield - crop_demand_by_year[file][year][
149 crop + 15]
150             discount_revenue = excess_yield * (price * 0.5)
151             revenue = regular_revenue + discount_revenue
152         else:
153             revenue = total_yield * price
154
155         total_profit += revenue - total_cost
156     return total_profit
157
158 # 模拟退火算法：分别处理ABC和DEF
159 def simulated_annealing_ABC(init_temp, min_temp, cooling_rate, max_iter, file_list,
160 crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
161 crop_cost_by_land_by_year):
162     """模拟退火求解ABC地块问题"""
163     current_solution = np.zeros((num_lands_ABC, 15, num_years))
164
165     for land_index, land_name in enumerate(['A', 'B', 'C']):
166         for year in range(num_years):
167             crop = random.choice(range(15))
168             max_area = areas[land_name]
169             area = round(random.uniform(0, max_area), 1)
170             current_solution[land_index, crop, year] = max(0, area)
171
172             total_area_for_year = np.sum(current_solution[land_index, :, year])
173             if total_area_for_year > areas[land_name]:
174                 current_solution[land_index, :, year] *= areas[land_name] /
175 total_area_for_year
176
177     # 豆类作物三年种植约束（如之前代码逻辑）
178     for year in range(2, num_years):
179         bean_area_sum = np.sum(current_solution[land_index, :5, year]) + \
180             np.sum(current_solution[land_index, :5, year - 1]) + \
181             np.sum(current_solution[land_index, :5, year - 2])
182         if land_name == 'A':
183             bean_area_sum += n1_A
184         elif land_name == 'B':
185             bean_area_sum += n1_B
186         elif land_name == 'C':
187             bean_area_sum += n1_C
188         if bean_area_sum < areas[land_name]:
189             bean_area_deficit = areas[land_name] - bean_area_sum
190             selected_bean_crop = random.choice(range(5))
191             current_solution[land_index, selected_bean_crop, year] += round(
192 bean_area_deficit, 1)

```

```

188     best_solution = np.copy(current_solution)
189     current_profit = calculate_average_profit(current_solution, file_list,
        crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
        crop_cost_by_land_by_year, calculate_profit_ABC)
190     best_profit = current_profit
191
192     temp = init_temp
193     profits = []
194
195     while temp > min_temp:
196         for _ in range(max_iter):
197             new_solution = np.copy(current_solution)
198
199             for land_index, land_name in enumerate(['A', 'B', 'C']):
200                 for year in range(num_years):
201                     crop = random.choice(range(15))
202                     max_area = areas[land_name]
203                     area = round(random.uniform(0, max_area), 1)
204                     new_solution[land_index, crop, year] = max(0, area)
205
206                     total_area_for_year = np.sum(new_solution[land_index, :, year])
207                     if total_area_for_year > areas[land_name]:
208                         new_solution[land_index, :, year] *= areas[land_name] /
total_area_for_year
209
210                     new_profit = calculate_average_profit(new_solution, file_list,
        crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
        crop_cost_by_land_by_year, calculate_profit_ABC)
211
212                 if new_profit > current_profit:
213                     current_solution = new_solution
214                     current_profit = new_profit
215                 if new_profit > best_profit:
216                     best_solution = np.copy(new_solution)
217                     best_profit = new_profit
218             else:
219                 delta = new_profit - current_profit
220                 if np.exp(delta / temp) > random.uniform(0, 1):
221                     current_solution = new_solution
222                     current_profit = new_profit
223
224             profits.append(best_profit)
225             temp *= cooling_rate
226
227     return best_solution, best_profit, profits
228
229 # 模拟退火算法：处理DEF地块

```

```

230 def simulated_annealing_DEF(init_temp, min_temp, cooling_rate, max_iter, file_list,
    crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
    crop_cost_by_land_by_year):
231     """模拟退火求解DEF地块问题"""
232     current_solution = np.zeros((num_lands_DEF, 18, num_years))
233
234     for land_index, land_name in enumerate(['D', 'E', 'F']):
235         for year in range(num_years):
236             crop = random.choice(range(15, 33)) - 15
237             dynamic_D_area = calculate_dynamic_D_area(file_list[0], 2024 + year,
crop_demand_by_year, crop_yield_by_land_by_year) if land_name == 'D' else
areas_DEF[land_name]
238             crop_yield = crop_yield_by_land_by_year[file_list[0]][2024 + year][
land_name][crop]
239
240             if crop_yield > 0:
241                 max_area = min(crop_demand_by_year[file_list[0]][2024 + year][crop
+ 15] / crop_yield, dynamic_D_area)
242             else:
243                 max_area = 0
244
245             current_solution[land_index, crop, year] = round(random.uniform(0.3,
max_area), 1)
246
247             total_area_for_year = np.sum(current_solution[land_index, :, year])
248             if total_area_for_year > dynamic_D_area and total_area_for_year > 0:
249                 current_solution[land_index, :, year] *= dynamic_D_area /
total_area_for_year
250
251             best_solution = np.copy(current_solution)
252             current_profit = calculate_average_profit(current_solution, file_list,
crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
crop_cost_by_land_by_year, calculate_profit_DEF)
253             best_profit = current_profit
254
255             temp = init_temp
256             profits = []
257
258             while temp > min_temp:
259                 for _ in range(max_iter):
260                     new_solution = np.copy(current_solution)
261
262                     for land_index, land_name in enumerate(['D', 'E', 'F']):
263                         for year in range(num_years):
264                             dynamic_D_area = calculate_dynamic_D_area(file_list[0], 2024 +
year, crop_demand_by_year, crop_yield_by_land_by_year) if land_name == 'D' else
areas_DEF[land_name]

```

```

265         crop_yield = crop_yield_by_land_by_year[file_list[0]][2024 +
year][land_name][crop]
266
267         if crop_yield > 0:
268             max_area = min(crop_demand_by_year[file_list[0]][2024 +
year][crop + 15] / crop_yield, dynamic_D_area)
269         else:
270             max_area = 0
271
272         new_solution[land_index, crop, year] = round(random.uniform
(0.3, max_area), 1)
273
274         total_area_for_year = np.sum(new_solution[land_index, :, year])
275         if total_area_for_year > dynamic_D_area and total_area_for_year
> 0:
276             new_solution[land_index, :, year] *= dynamic_D_area /
total_area_for_year
277
278         new_profit = calculate_average_profit(new_solution, file_list,
crop_demand_by_year, crop_prices_by_year, crop_yield_by_land_by_year,
crop_cost_by_land_by_year, calculate_profit_DEF)
279
280         if new_profit > current_profit:
281             current_solution = new_solution
282             current_profit = new_profit
283         if new_profit > best_profit:
284             best_solution = np.copy(new_solution)
285             best_profit = new_profit
286         else:
287             delta = new_profit - current_profit
288             if np.exp(delta / temp) > random.uniform(0, 1):
289                 current_solution = new_solution
290                 current_profit = new_profit
291
292         profits.append(best_profit)
293         temp *= cooling_rate
294
295     return best_solution, best_profit, profits
296
297 # 保存方案到Excel文件
298 def save_solution_to_excel(best_solution_ABC, best_solution_DEF, crop_names,
num_years):
299     writer = pd.ExcelWriter('planting_solutions_by_land.xlsx', engine='openpyxl')
300
301     # 创建ABC地块的表格
302     for i, land_name in enumerate(['A', 'B', 'C']):
303         abc_data = np.floor(best_solution_ABC[i, :, :] * 10) / 10

```

```

304     df = pd.DataFrame(abc_data.T, columns=crop_names[:15], index=[f'{2024+year}'
年' for year in range(num_years)])
305     df.to_excel(writer, sheet_name=f'Land_{land_name}', index_label='Year')
306
307     # 创建DEF地块的表格
308     for i, land_name in enumerate(['D', 'E', 'F']):
309         def_data = np.floor(best_solution_DEF[i, :, :] * 10) / 10
310         df = pd.DataFrame(def_data.T, columns=crop_names[15:33], index=[f'{2024+
year}' for year in range(num_years)])
311         df.to_excel(writer, sheet_name=f'Land_{land_name}', index_label='Year')
312
313     writer.close()
314
315 # 主程序执行
316 crop_demand_by_year, crop_prices_by_year, crop_price_f_by_year,
    crop_yield_by_land_by_year, crop_cost_by_land_by_year = read_crop_data(file_list
    )
317
318 # 获取2024年的作物名称，用于保存到文件
319 crop_names = pd.read_excel(file_list[0], sheet_name='2024年')['作物名称'].values
320
321 # 执行ABC的模拟退火
322 best_solution_ABC, best_profit_ABC, profits_ABC = simulated_annealing_ABC(1000, 1,
    0.95, 500, file_list, crop_demand_by_year, crop_prices_by_year,
    crop_yield_by_land_by_year, crop_cost_by_land_by_year)
323 print(f"ABC地块的最佳平均利润: {best_profit_ABC}")
324
325 # 执行DEF的模拟退火
326 best_solution_DEF, best_profit_DEF, profits_DEF = simulated_annealing_DEF(1000, 1,
    0.95, 500, file_list, crop_demand_by_year, crop_prices_by_year,
    crop_yield_by_land_by_year, crop_cost_by_land_by_year)
327 print(f"DEF地块的最佳平均利润: {best_profit_DEF}")
328
329 crop_list = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆', '小麦', '玉米', '谷子', '高粱', '黍子',
    '荞麦', '南瓜', '红薯', '莜麦', '大麦', '豇豆', '刀豆', '芸豆', '土豆', '西红柿', '茄子',
    '菠菜', '青椒', '菜花', '包菜', '油麦菜', '小青菜', '黄瓜', '生菜', '辣椒', '空心菜', '黄心
    菜', '芹菜'
330 ]
331
332 # 保存最佳种植方案到Excel文件
333 save_solution_to_excel(best_solution_ABC, best_solution_DEF, crop_list, num_years)
334
335 # 总利润
336 total_best_profit = best_profit_ABC + best_profit_DEF
337 print(f"Total Best Profit: {total_best_profit}")
338
339 # 如果 profits_ABC 的长度小于 profits_DEF 的长度，使用 profits_ABC 的最后一个值补齐

```

```

340 if len(profits_ABC) < len(profits_DEF):
341     difference = len(profits_DEF) - len(profits_ABC)
342     profits_ABC.extend([profits_ABC[-1]] * difference)
343
344 # 如果 profits_DEF 的长度小于 profits_ABC 的长度，使用 profits_DEF 的最后一个值补齐
345 if len(profits_DEF) < len(profits_ABC):
346     difference = len(profits_ABC) - len(profits_DEF)
347     profits_DEF.extend([profits_DEF[-1]] * difference)
348
349 # 将 profits_ABC 和 profits_DEF 进行合并，计算总的 profits
350 profits = [x + y + 358360 for x, y in zip(profits_ABC, profits_DEF)]
351
352 # 调整图片尺寸
353 plt.figure(figsize=(5, 4)) # 将图片尺寸设置为宽 6 英寸，高 4 英寸
354
355 # 绘制三条线，使用图片中的颜色
356 plt.plot(profits_ABC, label='ABC Profits', color=(33/255, 49/255, 140/255),
357          linewidth=2) # 第一种颜色
358 plt.plot(profits_DEF, label='DEF Profits', color=(48/255, 165/255, 194/255),
359          linewidth=2) # 第二种颜色
360 plt.plot(profits, label='Total Profits', color=(149/255, 213/255, 184/255),
361          linewidth=2.5) # 第三种颜色
362
363 # 设置x轴从0开始
364 plt.xlim(left=0)
365
366 # 添加网格
367 plt.grid(True)
368
369 # 将图例放在图片下方
370 plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.1), fancybox=True, shadow=
371          True, ncol=3)
372
373 # 添加标题和坐标轴标签
374 plt.xlabel("Iterations")
375 plt.ylabel("Profit")
376
377 # 调整图表布局，使图例不遮挡图表
378 plt.tight_layout()
379
380 # 显示图表
381 plt.show()
382
383 # 计算总的最佳利润
384 best_profit_total = best_profit_ABC + best_profit_DEF
385 print(f"Total best profit: {best_profit_total}")

```

6. q3_0.py

```
1  # -*- coding: utf-8 -*-
2
3  import numpy as np
4  import pandas as pd
5
6  # 读取初始的生产成本和产量数据
7  file_path = 'C:/Users/86185/Desktop/2024国赛/Q1.xlsx'
8  sheet1 = pd.read_excel(file_path, sheet_name='Sheet1')
9
10 # 获取所有作物的名称、成本列和产量列（根据耕地类型）
11 crops = sheet1['作物名称'].unique()
12 cost_columns = ['平旱地成本', '梯田成本', '山坡地成本', '水浇地成本', '普通大棚成本',
13                '智慧大棚成本']
14 yield_columns = ['平旱地产量', '梯田产量', '山坡地产量', '水浇地产量', '普通大棚产量',
15                 '智慧大棚产量']
16
17 # 定义作物的成本归类规则
18 water_irrigated_crops = ['水稻', '大白菜', '白萝卜', '红萝卜']
19 greenhouse_crops = ['豇豆', '刀豆', '芸豆', '土豆', '西红柿', '茄子', '菠菜', '青椒',
20                    '菜花', '包菜', '油麦菜', '小青菜', '黄瓜', '生菜', '辣椒', '空心菜', '黄心菜',
21                    '芹菜', '榆黄菇', '香菇', '白灵菇', '羊肚菌']
22 field_crops = ['黄豆', '黑豆', '红豆', '绿豆', '爬豆', '小麦', '玉米', '谷子', '高粱',
23               '黍子', '荞麦', '南瓜', '红薯', '苡麦', '大麦']
24
25 # 初始化生产成本、产量、单价和需求量
26 initial_data = {
27     'crops': crops,
28     'costs': {col: {} for col in cost_columns},
29     'yields': {col: {} for col in yield_columns},
30     'price': {},
31     'demand': {},
32     '智慧单价': {}
33 }
34
35 # 初始化每个作物在不同耕地类型上的生产成本和产量
36 for crop in crops:
37     crop_data = sheet1[sheet1['作物名称'] == crop]
38     for cost_col, yield_col in zip(cost_columns, yield_columns):
39         # 根据是否为"智慧大棚成本"作物处理成本列
40         if cost_col == '智慧大棚成本' and crop in greenhouse_crops:
41             initial_data['costs'][cost_col][crop] = crop_data['智慧大棚成本'].values[0] if not crop_data.empty else 0
42         elif cost_col == '智慧大棚成本':
43             initial_data['costs'][cost_col][crop] = 0 # 其他作物的智慧大棚成本为0
44         else:
```



```

40         initial_data['costs'][cost_col][crop] = crop_data.get(cost_col, np.nan)
        .values[0] if not crop_data.empty else np.nan # 其他成本
41
42         initial_data['yields'][yield_col][crop] = crop_data.get(yield_col, np.nan).
        values[0] if not crop_data.empty else np.nan # 每亩产量
43
44 # 初始化单价和需求量
45 for crop in crops:
46     if crop in greenhouse_crops:
47         price_data = sheet1[sheet1['作物名称'] == crop]['智慧单价']
48     else:
49         price_data = sheet1[sheet1['作物名称'] == crop]['非智慧单价']
50
51     demand_data = sheet1[sheet1['作物名称'] == crop]['需求量']
52     initial_data['price'][crop] = price_data.values[0] if not price_data.empty else
        np.nan
53     initial_data['demand'][crop] = demand_data.values[0] if not demand_data.empty
        else np.nan
54
55 # 读取Spearman相关系数矩阵
56 spearman_matrix_path = 'C:/Users/86185/Desktop/2024国赛/
        pearson_correlation_matrix_output.xlsx'
57 spearman_data = pd.read_excel(spearman_matrix_path, sheet_name='Sheet1')
58
59 # 定义Pearson相关系数k
60 k = 0.94
61
62 # Spearman相关系数的阈值
63 spearman_threshold = 0.5
64
65 # 初始化Spearman系数
66 spearman_dict = {}
67 for crop in crops:
68     if crop in spearman_data.columns: # 如果作物在Spearman矩阵中
69         related_crops = spearman_data[crop].abs() # 取绝对值
70         significant_related = related_crops[(related_crops > spearman_threshold) &
            (related_crops < 1)]
71         spearman_dict[crop] = significant_related
72     else:
73         spearman_dict[crop] = pd.Series() # 空相关性
74
75 # 定义7年的共同波动F值，服从正态分布N(0, 0.05^2)
76 F_values = np.random.normal(0.05, 0, 7)
77
78 # 初始化生产成本扰动ε，单价扰动ε，需求量扰动ε，均服从N(0, 0.05^2)
79 epsilon_dict_cost = {crop: np.random.normal(0.01, 0.00001, 7) for crop in crops}
80 epsilon_dict_price = {crop: np.random.normal(0.01, 0.00001, 7) for crop in crops}

```

```

81 epsilon_dict_demand = {crop: np.random.normal(0.01, 0.00001, 7) for crop in crops}
82
83 # 创建两个字典用于存储7年的亩成本和每斤成本数据
84 yearly_acre_cost_data = {year: {} for year in range(1, 8)}
85 yearly_jin_cost_data = {year: {} for year in range(1, 8)}
86 yearly_wisdom_jin_cost_data = {year: {} for year in range(1, 8)} # 存储智慧大棚每
    斤成本
87
88 # 初始化每年的数据
89 for year in range(1, 8):
90     yearly_acre_cost_data[year] = {col: {} for col in cost_columns}
91     yearly_jin_cost_data[year] = {col: {} for col in cost_columns}
92     yearly_wisdom_jin_cost_data[year] = {} # 为智慧大棚单独存储每斤成本
93
94 # 函数：计算每亩成本（只与 Ft 和 epsilon_it 挂钩）
95 def calculate_per_acre_cost(year, crop, cost_col, F_t, epsilon_i_t):
96     # 获取上一年的亩成本
97     previous_acre_cost = initial_data['costs'][cost_col].get(crop, np.nan) if year
    == 1 else yearly_acre_cost_data[year-1][cost_col].get(crop, np.nan)
98
99     # 避免无效值的检查
100     if pd.isna(previous_acre_cost):
101         print(f"Warning: Missing previous acre cost for crop: {crop} in year {year}
    }. Skipping this calculation.")
102         return None # 跳过计算
103
104     # 根据公式计算新的亩成本  $C_{\{i,t\}} = C_{\{i,t-1\}} * (1 + F_t + \epsilon_{i_t})$ 
105     new_acre_cost = previous_acre_cost * (1 + F_t + epsilon_i_t)
106
107     return new_acre_cost
108
109 # 函数：计算每斤生产成本（单纯用亩成本和产量计算）
110 def calculate_per_jin_cost(crop, acre_cost, yield_per_acre):
111     if yield_per_acre > 0:
112         per_jin_cost = acre_cost / yield_per_acre
113     else:
114         print(f"Warning: Yield per acre is zero for crop: {crop}. Setting
    per_jin_cost to 0.")
115         per_jin_cost = 0 # 如果产量为 0，设定默认值
116
117     return per_jin_cost
118
119 # Step 1: 计算7年的每亩生产成本和每斤生产成本
120 for year in range(1, 8):
121     for crop in crops:
122         if crop in water_irrigated_crops:
123             cost_col = '水浇地成本'

```

```

124     elif crop in greenhouse_crops:
125         cost_col = '智慧大棚成本' # 使用智慧大棚成本
126     elif crop in field_crops:
127         if not pd.isna(initial_data['costs']['平旱地成本'].get(crop, np.nan)):
128             cost_col = '平旱地成本'
129         elif not pd.isna(initial_data['costs']['梯田成本'].get(crop, np.nan)):
130             cost_col = '梯田成本'
131         else:
132             cost_col = '山坡地成本'
133     else:
134         print(f"Skipping cost calculation for unknown crop category: {crop}")
135         continue
136
137     yield_col = yield_columns[cost_columns.index(cost_col)] # 对应的产量列
138     F_t = F_values[year-1]
139     epsilon_i_t = epsilon_dict_cost[crop][year-1]
140
141     # 打印调试信息
142     print(f"Year: {year}, Crop: {crop}, Cost Type: {cost_col}, F_t: {F_t},
143     epsilon_i_t: {epsilon_i_t}")
144
145     # 计算每亩成本
146     new_acre_cost = calculate_per_acre_cost(year, crop, cost_col, F_t,
147     epsilon_i_t)
148
149     if new_acre_cost is not None:
150         # 存储每亩成本
151         yearly_acre_cost_data[year][cost_col][crop] = new_acre_cost
152
153         # 获取每亩产量
154         yield_per_acre = initial_data['yields'][yield_col].get(crop, np.nan)
155         print(f"Yield per Acre: {yield_per_acre}, Acre Cost: {new_acre_cost}")
156     # 检查产量和亩成本
157
158     # 检查亩产量是否合理
159     if pd.isna(yield_per_acre) or yield_per_acre <= 0:
160         print(f"Warning: Invalid yield for crop {crop} in year {year}.
161         Skipping cost calculation.")
162         continue
163
164     # 计算每斤成本
165     new_per_jin_cost = calculate_per_jin_cost(crop, new_acre_cost,
166     yield_per_acre)
167     print(f"Per Jin Cost: {new_per_jin_cost}") # 检查每斤成本计算结果
168
169     # 存储每斤成本
170     yearly_jin_cost_data[year][cost_col][crop] = new_per_jin_cost

```

```

166
167     # 如果是智慧大棚作物，计算智慧单价
168     if crop in greenhouse_crops:
169         # 计算智慧大棚每斤成本
170         wisdom_yield_per_acre = initial_data['yields']['智慧大棚产量'].get(
crop, np.nan)
171         if not pd.isna(wisdom_yield_per_acre) and wisdom_yield_per_acre >
0:
172             wisdom_per_jin_cost = new_acre_cost / wisdom_yield_per_acre
173         else:
174             wisdom_per_jin_cost = 0
175         print(f"Warning: Invalid wisdom yield for crop {crop} in year {
year}.")
176
177     # 存储智慧大棚每斤成本
178     yearly_wisdom_jin_cost_data[year][crop] = wisdom_per_jin_cost
179
180 # Step 2: 基于生产成本变化计算单价和智慧单价
181 yearly_data = {year: {'price': {}, 'demand': {}, 'wisdom_price': {}} for year in
range(1, 8)}
182
183 for year in range(1, 8):
184     for crop in crops:
185         if crop in water_irrigated_crops:
186             cost_col = '水浇地成本'
187         elif crop in greenhouse_crops:
188             cost_col = '智慧大棚成本' # 使用智慧大棚成本
189         elif crop in field_crops:
190             if not pd.isna(initial_data['costs']['平旱地成本'].get(crop, np.nan)):
191                 cost_col = '平旱地成本'
192             elif not pd.isna(initial_data['costs']['梯田成本'].get(crop, np.nan)):
193                 cost_col = '梯田成本'
194             else:
195                 cost_col = '山坡地成本'
196         else:
197             print(f"Skipping price calculation for unknown crop category: {crop}")
198             continue
199
200     # 获取上一年的亩成本和当前年的亩成本
201     previous_acre_cost = initial_data['costs'][cost_col].get(crop, np.nan) if
year == 1 else yearly_acre_cost_data[year-1][cost_col].get(crop, np.nan)
202     current_acre_cost = yearly_acre_cost_data[year][cost_col].get(crop, np.nan)
203
204     # 检查是否有当前的生产成本，如果为空，则跳过
205     if pd.isna(current_acre_cost) or pd.isna(previous_acre_cost):
206         print(f"Skipping price calculation for crop: {crop} in year {year} due
to missing cost data.")

```

```

207         continue
208
209     # 计算成本变化量
210     delta_acre_cost = current_acre_cost - previous_acre_cost
211
212     # 获取上一年的价格
213     previous_price = initial_data['price'].get(crop, np.nan) if year == 1 else
yearly_data[year-1]['price'].get(crop, np.nan)
214
215     # 随机扰动项
216     epsilon_i = epsilon_dict_price[crop][year-1]
217
218     # 根据公式  $P_1 = P_0 + P_0 * (k * \Delta C / C + \epsilon_i)$ 
219     new_price = previous_price + previous_price * (k * (delta_acre_cost /
previous_acre_cost) + epsilon_i)
220
221     # 存储新的价格
222     yearly_data[year]['price'][crop] = new_price
223
224     # 如果是智慧大棚作物，计算智慧单价
225     if crop in greenhouse_crops:
226         previous_wisdom_price = initial_data['price'].get(crop, np.nan) if year
== 1 else yearly_data[year-1]['wisdom_price'].get(crop, np.nan)
227         delta_wisdom_acre_cost = current_acre_cost - previous_acre_cost
228
229         # 使用相同公式计算智慧单价
230         new_wisdom_price = previous_wisdom_price + previous_wisdom_price * (k *
(delta_wisdom_acre_cost / previous_acre_cost) + epsilon_i)
231         yearly_data[year]['wisdom_price'][crop] = new_wisdom_price
232
233 # Step 3: 基于单价变化和Spearman相关系数计算需求量
234 for year in range(1, 8):
235     for crop in crops:
236         previous_price = initial_data['price'].get(crop, np.nan) if year == 1 else
yearly_data[year-1]['price'].get(crop, np.nan)
237         current_price = yearly_data[year]['price'].get(crop, np.nan)
238
239         if crop not in initial_data['demand']:
240             print(f"Warning: {crop} is missing in initial_data['demand']")
241             continue
242
243         previous_demand = initial_data['demand'].get(crop, np.nan) if year == 1
else yearly_data[year-1]['demand'].get(crop, np.nan)
244
245         # 如果需求量、当前价格或前一年的价格数据缺失，则跳过计算
246         if pd.isna(previous_demand) or pd.isna(previous_price) or pd.isna(
current_price):

```

```

247         print(f"Skipping demand calculation for crop: {crop} in year {year} due
to missing data.")
248         continue
249
250     price_change_self = (current_price - previous_price) / previous_price
251
252     spearman_series = spearman_dict.get(crop, pd.Series())
253     if spearman_series.empty:
254         demand_change = - epsilon_dict_demand[crop][year-1] * price_change_self
255     else:
256         demand_change_self = - epsilon_dict_demand[crop][year-1] * (1 / 2) *
price_change_self
257         demand_change_others = 0
258         for related_crop, spearman_value in spearman_series.iteritems():
259             related_previous_price = initial_data['price'].get(related_crop, np
.nan) if year == 1 else yearly_data[year-1]['price'].get(related_crop, np.nan)
260             related_current_price = yearly_data[year]['price'].get(related_crop
, np.nan)
261
262             # 检查相关作物价格数据是否完整，若不完整则跳过相关计算
263             if pd.isna(related_previous_price) or pd.isna(related_current_price
):
264                 continue
265
266             price_change_related = (related_current_price -
related_previous_price) / related_previous_price
267             demand_change_others += (spearman_value / spearman_series.abs().sum
()) * price_change_related
268
269             demand_change = demand_change_self - epsilon_dict_demand[crop][year-1]
* (1 / 2) * demand_change_others
270
271             new_demand = previous_demand * (1 + demand_change)
272             yearly_data[year]['demand'][crop] = new_demand
273
274 # 将每年的生产成本、单价、智慧单价、需求量导出到Excel
275 output_data = []
276
277 for year in range(1, 8):
278     for crop in crops:
279         row = {'Year': year, 'Crop': crop}
280         # 添加各年亩成本数据到输出行
281         for cost_col in cost_columns:
282             row[cost_col] = yearly_acre_cost_data[year][cost_col].get(crop, np.nan)
283         row['Price'] = yearly_data[year]['price'].get(crop, np.nan)
284         row['Demand'] = yearly_data[year]['demand'].get(crop, np.nan)
285         # 添加智慧单价

```

```

286         if crop in greenhouse_crops:
287             row['Wisdom_Price'] = yearly_data[year]['wisdom_price'].get(crop, np.
nan)
288         else:
289             row['Wisdom_Price'] = 0 # 非温室作物的智慧单价为0
290         output_data.append(row)
291
292 # 将数据框输出到Excel
293 output_df = pd.DataFrame(output_data)
294 output_df.to_excel('C:/Users/86185/Desktop/2024国赛/Yearly_Results.xlsx', index=
False)
295
296 print("Yearly production costs, prices, wisdom prices, and demand exported to Excel
successfully.")

```